II – CONCEPTS DE BASE DE LA MODELISATION ORIENTEE OBJETS

- II-1 Classe
- II-2 Objet
- II-3 Encapsulation
- II-4 Association, Agrégation et Composition entre classes
- II-5 Multiplicité
- II-6 Héritage
- II-7 Généralisation/Spécialisation entre classes
- II-8 Polymorphisme

II-1 Classe

- **Une classe est une structure de données commune à plusieurs objets de la vie réelle.**

- **Elle factorise un ensemble d'attributs et de comportements d'un groupe d'objets;**

- **Elle définit ainsi un moule, un format commun pour les tous d'objets du groupe concerné;**

- **Une classe est un type abstrait caractérisé par des propriétés que sont: les attributs (qui stockent les données) et les méthodes (qui assurent les traitements souhaités sur ces données).**

Exemple de classe

Voiture (nom de la classe)

attributs: (données)

marque

couleur

nbrePortes

immatricul

méthodes: (traitements, comportements ou services)

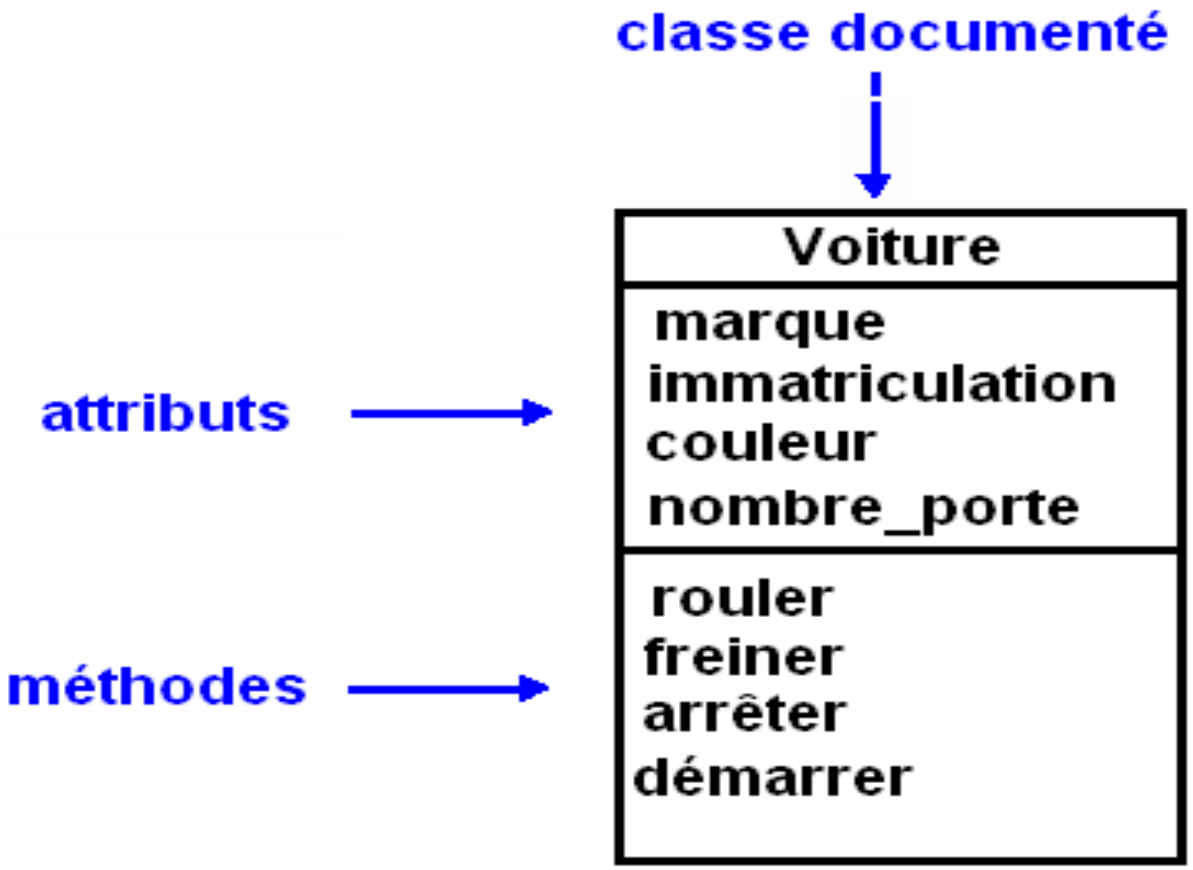
demarrer()

rouler()

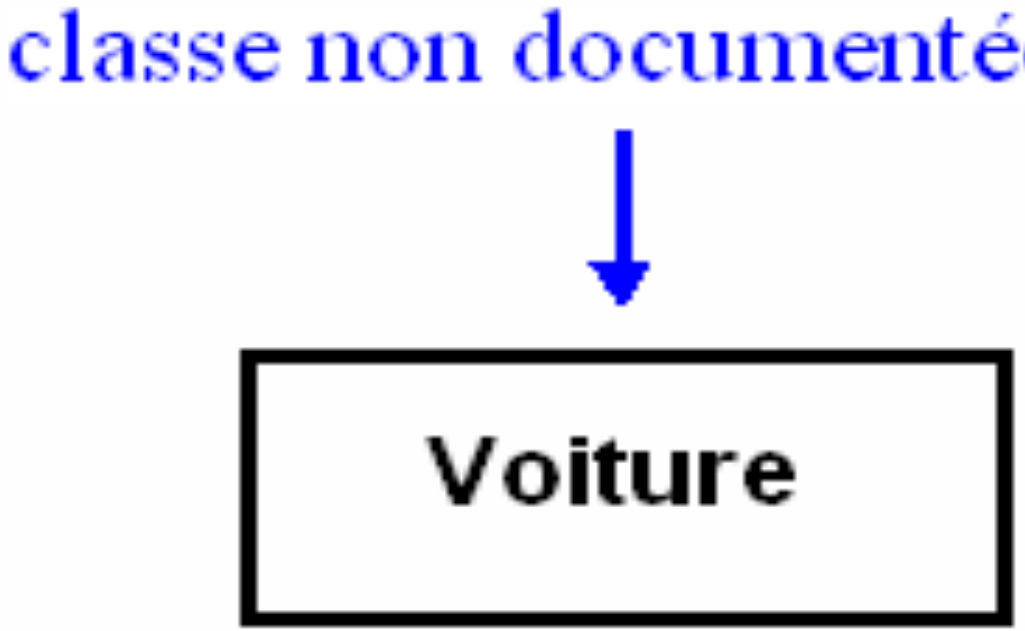
freiner()

s-arrêter().

Représentation de la classe Voiture



Représentation simplifiée de la classe Voiture



II-2 Objet

- **Un objet n'est alors rien d'autre qu'une instance de classe; c-à-d une classe dont les attributs sont renseignés et une ou des méthodes sont mises en œuvre;**

- **Ainsi l'objet est une entité qui comprend:**
  - **un identificateur (son nom défini à partir du nom de la classe)**
  - **un état**
  - **un comportement;**

- **Exemple d'objet**

  Considérons la classe: Voiture;

  Nom objet :Voiture (ou Voiture ou encore Peugeot206 :Voiture)

  Exemple Etat: marque = **Peugeot206;**

  couleur **= bleue;**

  nbreportes **= 4;**

  immatricul **= DK – 1233 – AA**

  Comportement: rouler()

L'état d'un objet ?

- Est défini par l'ensemble des valeurs prises par ses attributs;

- L'état est en général variable, car les attributs peuvent prendre de nouvelles valeurs en fonction du comportement du système.

- NB: un objet dont les attributs sont statiques (non dynamiques) a un état invariable, constant.

Attributs: simple, dérivé, de classe, multivalué ou composé

Un attribut est toujours typé : Le type définit le domaine des valeurs que peut prendre un attribut.

- **Un attribut simple : reçoit une seule valeur; Exemple: l'attribut nomPersonne ne reçoit qu'une seule valeur de type chaine (string, varchar, …);**

- **Un attribut dérivé : Sa valeur est tirée d'autres attributs de sa classe; on le note: /attribut (slash suivi du nom de l'attribut); Exemple : /prixTotal = prixHorsTaxe x Tva;**

- **Un attribut de classe (dit aussi *static*): est un attribut réservé uniquement à la classe; il n'est pas instancié dans les objets; il est renseigné pour la classe; on peut y accéder avec le nom de la classe s'il est public, sinon avec le getter associé qui devrait aussi être *static* et *public*; il est souligné, Exemple: l'attribut nbVoiture pour la classe Voiture;**

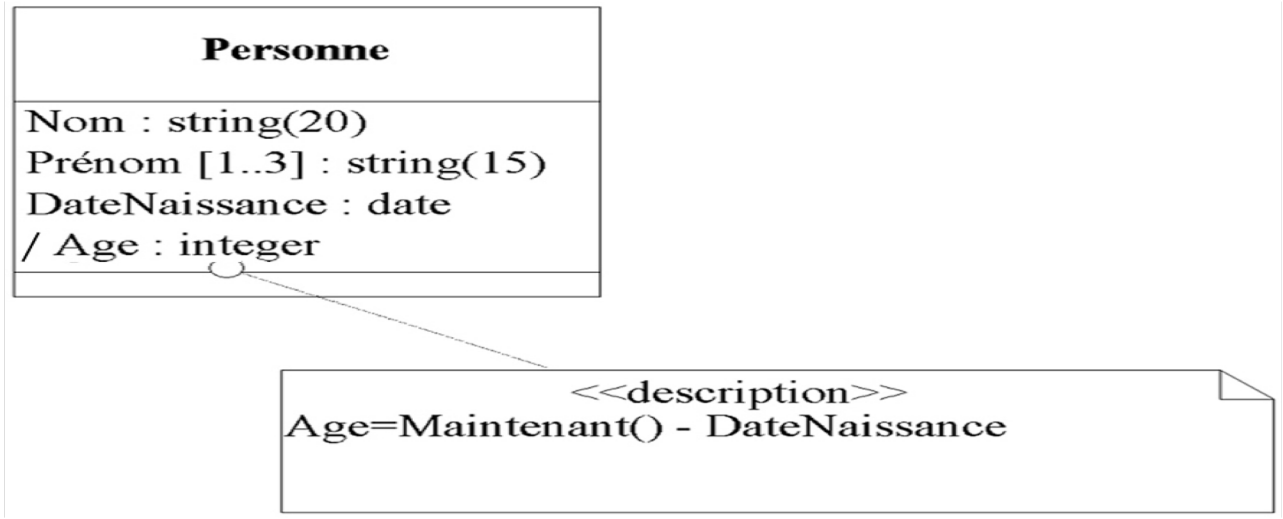
- **Un attribut peut être multivalué : Il peut prendre plusieurs valeurs distinctes de même type.**

Exemple: un attribut prenom multivalué peut enregistrer plusieurs prénoms d'une même personne, tels que Ndeye Fatou Bintou ; (en C++: un tableau de chaines; string prenom[3]);

- **Un attribut peut être composé (ou composite) : Il peut prendre plusieurs valeurs de types distincts; Il joue alors le rôle d'un groupe d'attributs.**

Exemple: un attribut adresse composite peut prendre nomVille, nomQuartier , rue, numeroVilla; (type: peut être classe Adresse).

Autres exemples: d'attributs simple, dérivé, multivalué ou composé



Age précédé d'un slash (/) est un attribut **dérivé**, il est calculé à partir de l'attribut DateNaissance et et de la dateActuelle du système.

Les autres attributs sont des attributs **simples**: nom et dateNaissance

Prénom est un attribut **multivalué**; ici une personne peut avoir 1 à 3 prénoms de longueur 15 caractères maximum chacun.

On pouvait ajouter l'attribut composite **adressPers** de **Adresse** une classe ayant pour attributs: **ville, quartier, rue, numVilla.**

Contraintes d'attribut ou de méthode

- ***{readOnly}*** : la valeur initiale de l'attribut ne peut plus être modifiée; on ne peut que la lire (équivalent à un **attribut constant**).

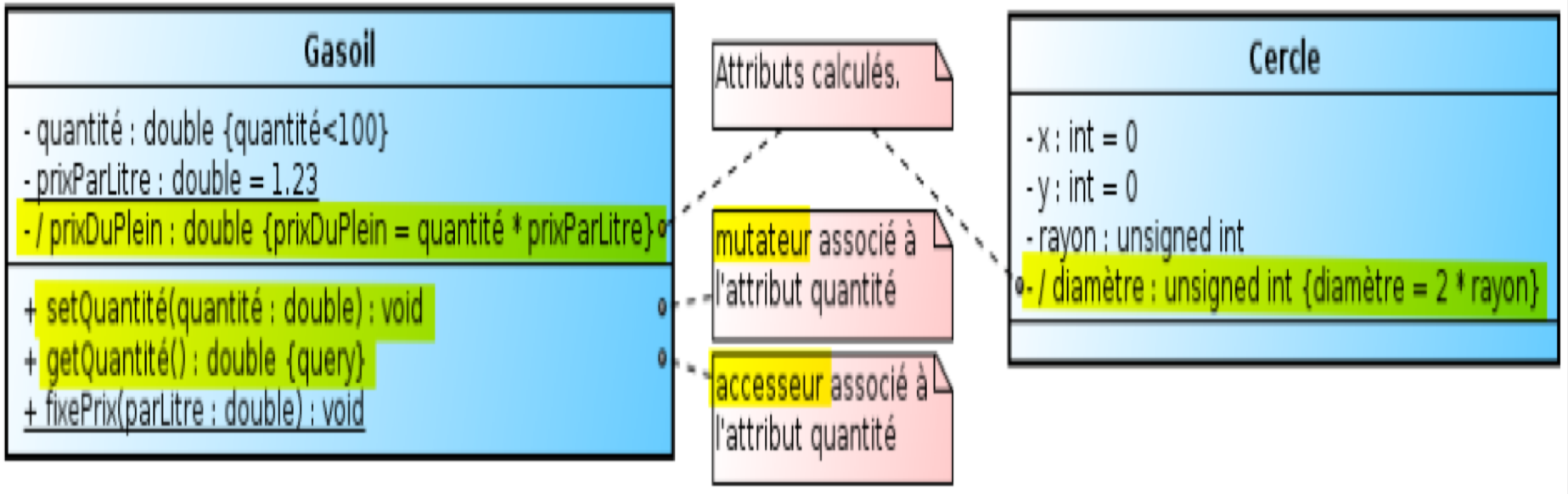
- ***{unique}*** : on demande cette fois-ci qu'il n'y ait aucun doublon dans les valeurs de l'attribut ciblé d'une même classe.

- ***{not null}*** : L'attribut doit à tout prix être initialisé (nécessaire dans le cas des pointeurs en C++)

- ***{query}*** : une méthode peut être déclarée comme étant de type requête (***query***); ainsi son exécution ne doit nullement modifier l'état de l'objet (ie des valeurs de ses attributs); utile pour la consultation;

- ***{libellé autre contraintes}*** : tel que Domaine_de_définition ou formule_de_renseignement d'un attribut d'un attribut dérivé, entre autres.

Exemple de contraintes d'attributs



Le comportement d'un objet

- Il correspond à la mise en oeuvre d'une des compétences (méthodes, appelées aussi fonctions, opérations, traitement ou service) de l'objet;

- Le comportement est déclenché par un stimuli, un appel (message) interne ou externe à l'objet.

- ***Exemple***

  rouler() défini un comportement de l'objet peugeot206; freiner() aussi.

Droits d'accès aux membres c-à-d propriétés (attributs et méthodes) d'une classe

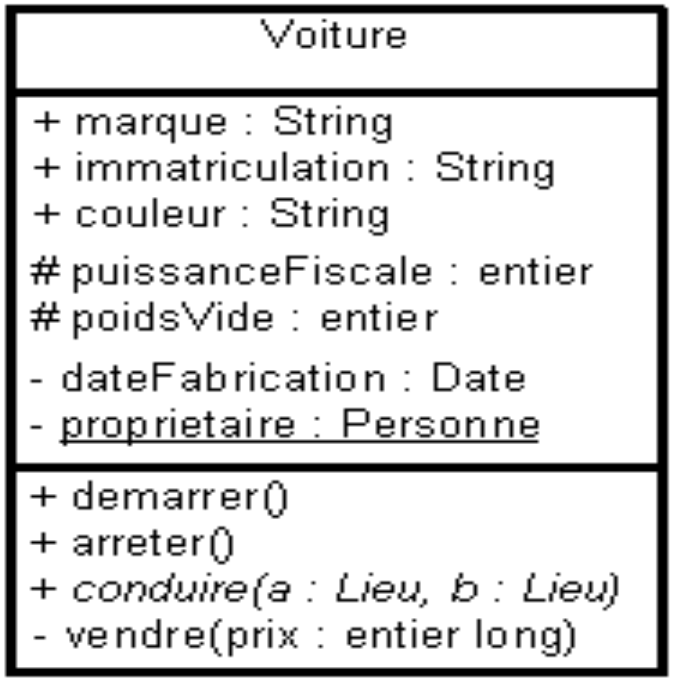
4 niveaux de visibilité:

- **public (+): si le membre est accessible partout dans le programme;**
- **private (-) : si le membre n'est accessible qu'aux membres de la même classe;**
- **protected (#) : si le membre n'est accessible qu'aux membres de la même classe et à ceux de ses sous-classes;**
- **Package (~) : si le membre n'est accessible qu'au sein du même package.**

NB: pour la notion de sous-classe, voir détails plus loin.

Représentation détaillée de la classe Voiture

classe détaillée: les attributs sont typés, les prototypes des méthodes spécifiés et les niveaux de protection des membres sont renseignés



Encapsulation

- L'encapsulation consiste à masquer les détails d'implémentation d'un objet; les attributs d'une classe sont encapsulés, c-à-d définis dans celle-ci.

- l'encapsulation facilite l'évolution d'une application car elle stabilise l'utilisation des objets;

- l'encapsulation garantit l'intégrité des données, car elle permet d'interdire l'accès direct aux attributs des objets (utilisation d'accesseurs et de mutateurs);

- la vue externe d'un objet correspond au service public offert par l'objet (c-à-d: à l'ensemble des propriétés publiques dont les méthodes publique de l'objet).

Direction des arguments d'une méthode

- On peut indiquer l'un des mots clés suivant devant le nom d'un argument d'une méthode:
  - **in**: La valeur de l'argument concerné est transmise à l'appel de la méthode par l'appelant de celle-ci (c'est le comportement par défaut si aucune direction n'est spécifiée).
  - **out** : La valeur finale de l'argument concerné est transmise au retour de l'appel de la méthode à l'appelant de celle-ci.
  - **inout** : La valeur de l'argument concerné est transmise à l'appel et au retour.

Exemple de direction d'arguments avec la classe Horloge

Considérons les méthodes: ***setTime*** *et* ***getTime*** *de la classe* ***Horloge:***

- **Prototypes**:
  - ***setTime (heures : int, minutes : int) : void***
  - ***getTime (heures : int, minutes : int) : void***

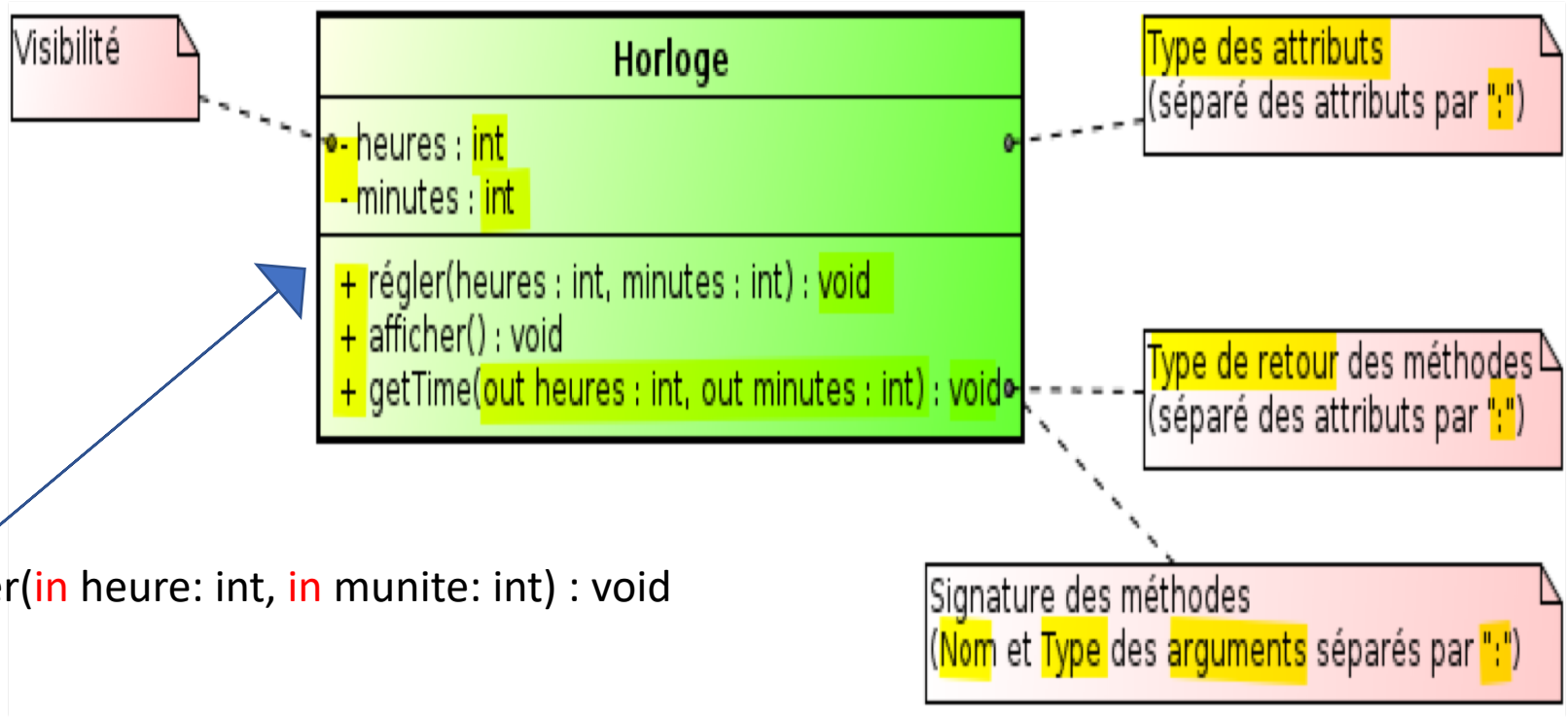
- Pour signifier que la méthode ***SetTime*** reçoit, à son appel, des arguments: les valeurs de ***heures*** et ***minutes***, on utilise le mot clé ***in*** pour indiquer la direction **entrant** des **arguments** comme suit : ***SetTime (in heures : int, in minutes : int) : void***

- De même, pour indiquer que la méthode ***getTime*** ne reçoit pas mais retourne les valeurs des attributs ***heures*** et ***minutes,*** on utilise le mot clé **out** pour indiquer la direction **sortant** des arguments : ***getTime(out heures : int, out minutes : int) : void.***

- s'il arrive, pour une fonction (méthode) donnée, de devoir recevoir des valeurs d'arguments à l'appel et retourner les « nouvelles » valeurs après exécution, c'est alors le mot clé **inout** qui utilisé sous les mêmes formes. **Par exemple**: en fusionnant **setTime** et **getTime** avec ***setgetTime (inout heures : int, inout minutes : int) : void***

- **NB: Si** on avait la fonction ***getHour*** qui ne renvoie que ***heures (une seule valeur)***, alors on aurait l'équivalence: ***getHour(heures : int) : int*** et ***getHour(out heures : int) : void.***

(Suite) Exemple précédent : classe Horloge détaillée avec direction d'arguments



regler(in heure: int, in munite: int) : void

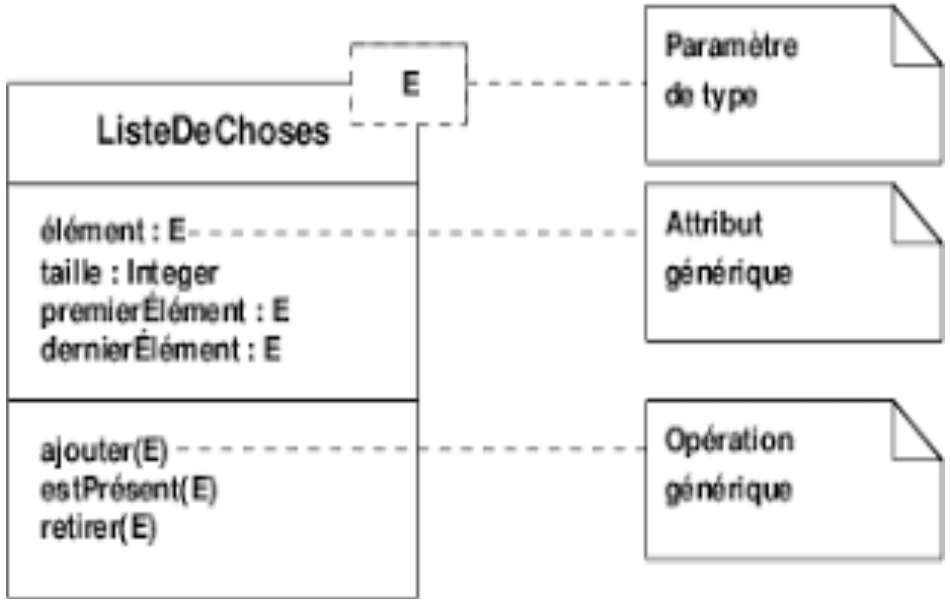
Fonction abstraite, Classe abstraite et Interface

- **Fonction (ou Méthode) abstraite**
  - C'est une fonction sans corps; elle ne peut s'exécuter; son **mode d'emploi** est défini par les fonctions héritières des classes dérivées (la **redéfinition**);
- **Classe abstraite**
  - Une classe est dite **abstraite dès qu'elle possède au moins une fonction abstraite**, contrairement aux **classes concrètes**; elle ne peut être instanciée; elle est en ***italique*** ou marquée du **stéréotype <<abstract>>**;
- **Interface**
  - Une interface "est une **classe abstraite** » dont **toutes les méthodes sont abstraites** et ne comportant**, si elle en a, que des attributs constants;** elle est stéréotypée **<<interface>>**;
  - Elle signe un **contrat de réalisation** (ie d'héritage et de définition complète de ces méthodes) avec les classes qui l'implémentent c-à-d qui la réalisent.

Classe paramétrée ou générique (Template)

- On appelle **Template** une classe dont les **attributs** et/ou **méthodes** (*arguments* et *retour*) sont typés génériquement;

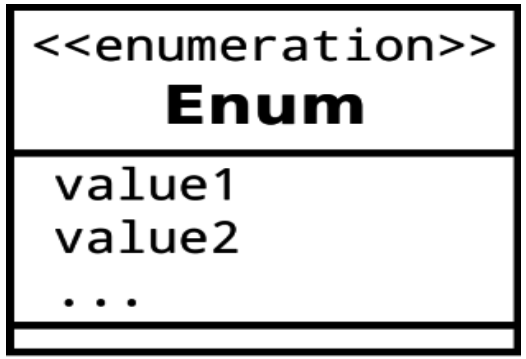
- **Exemple:**

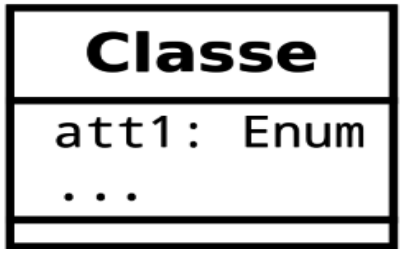


Stéréotype <<enumeration>>

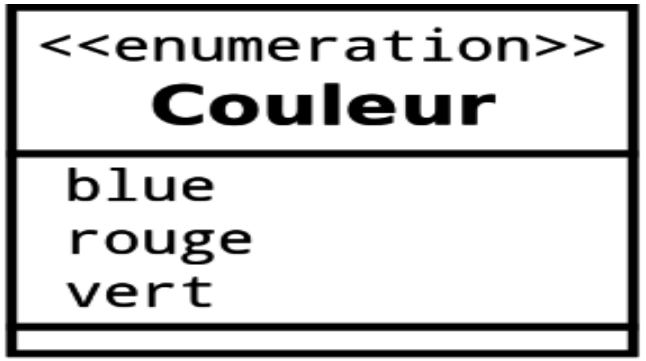
Permet de modéliser une liste déroulante représentant un liste de valeurs possibles pour un attribut

Syntaxe



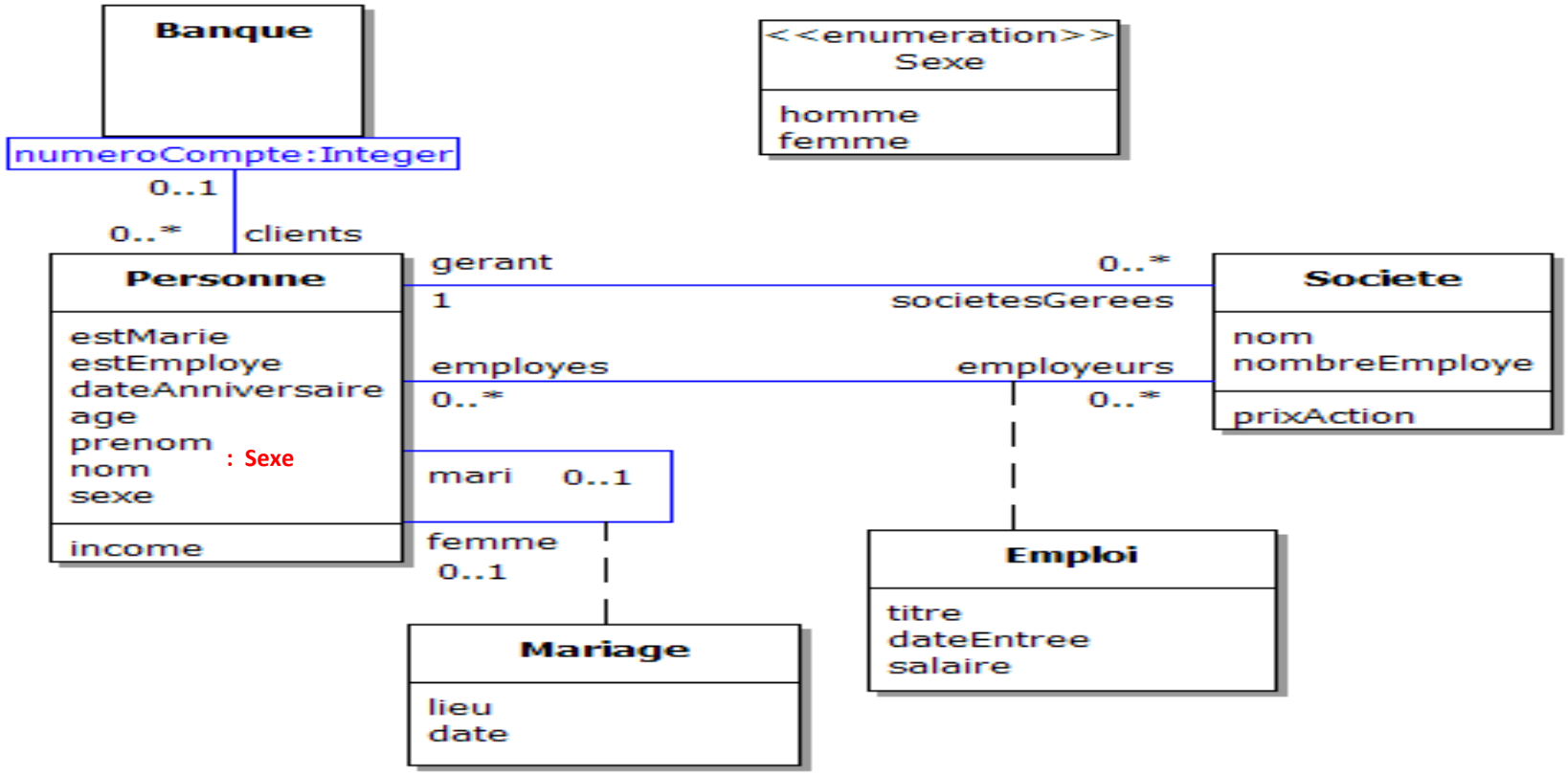


Exemple





Exemple d'utilisation: Stéréotype <<enumeration>>

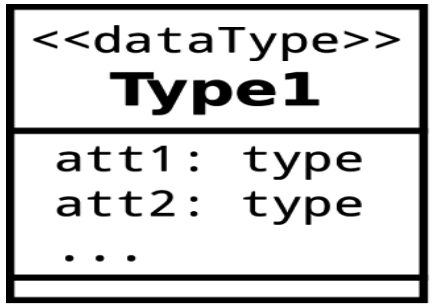


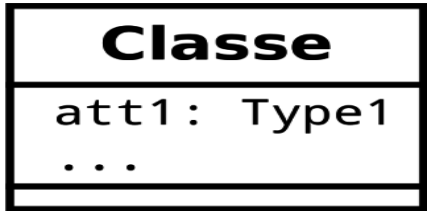
ATTENTION: la multiplicité côté Personne dans sa relation avec Banque est plutôt 0..1 et non 0..* (voir contrainte d'association: QUALIFICATION.)

Stéréotype <<dataType>>

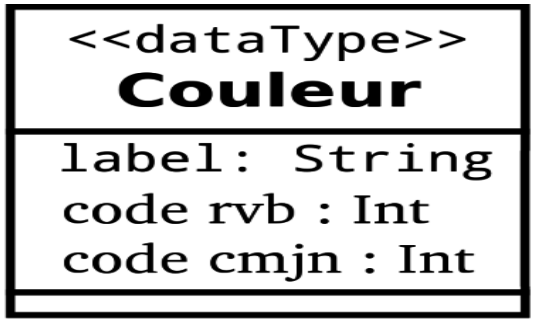
Permet de modéliser un attribut composé

Syntaxe





Exemple





NB:** L'attribut composé peut aussi être modélisé par une **Classe

II-4 Association, Agrégation et Composition entre classes

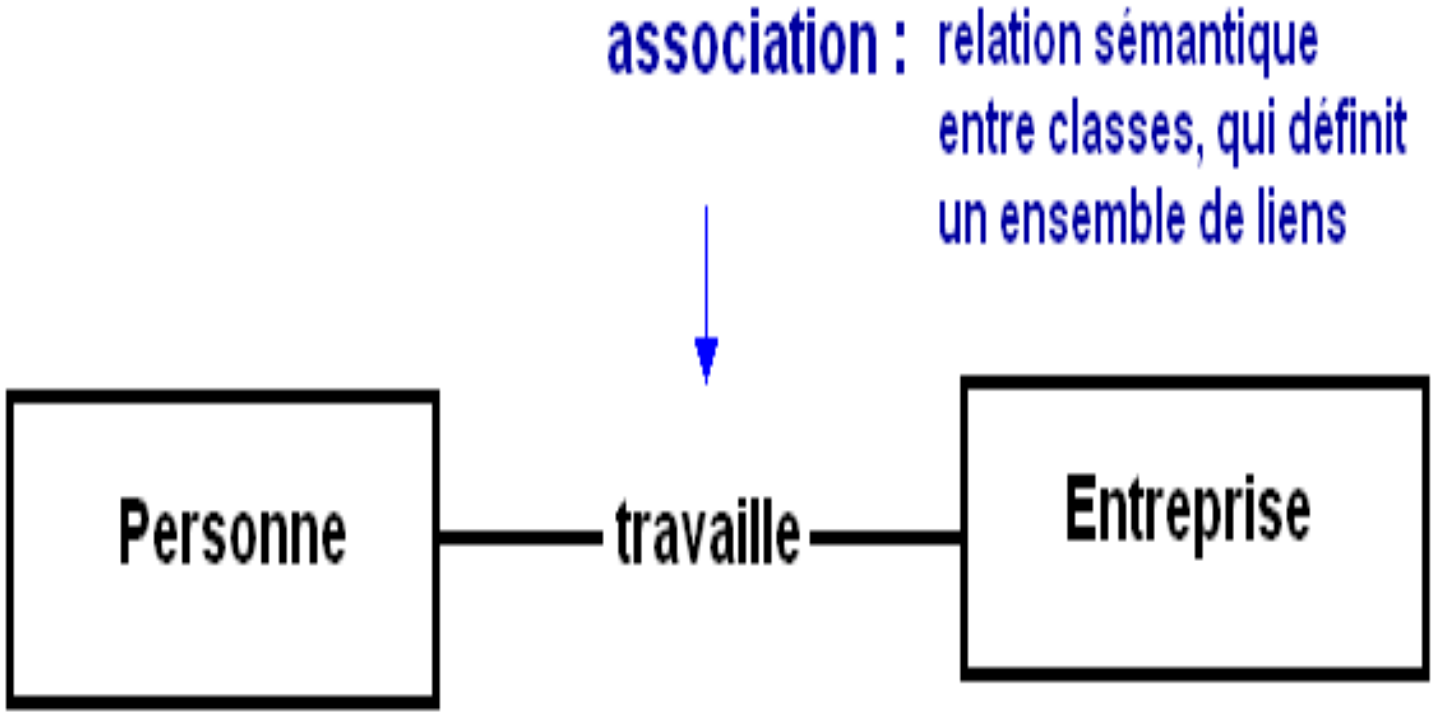
Ces concepts cristallisent différents liens entre objets de classes données.

II-4-1 Association

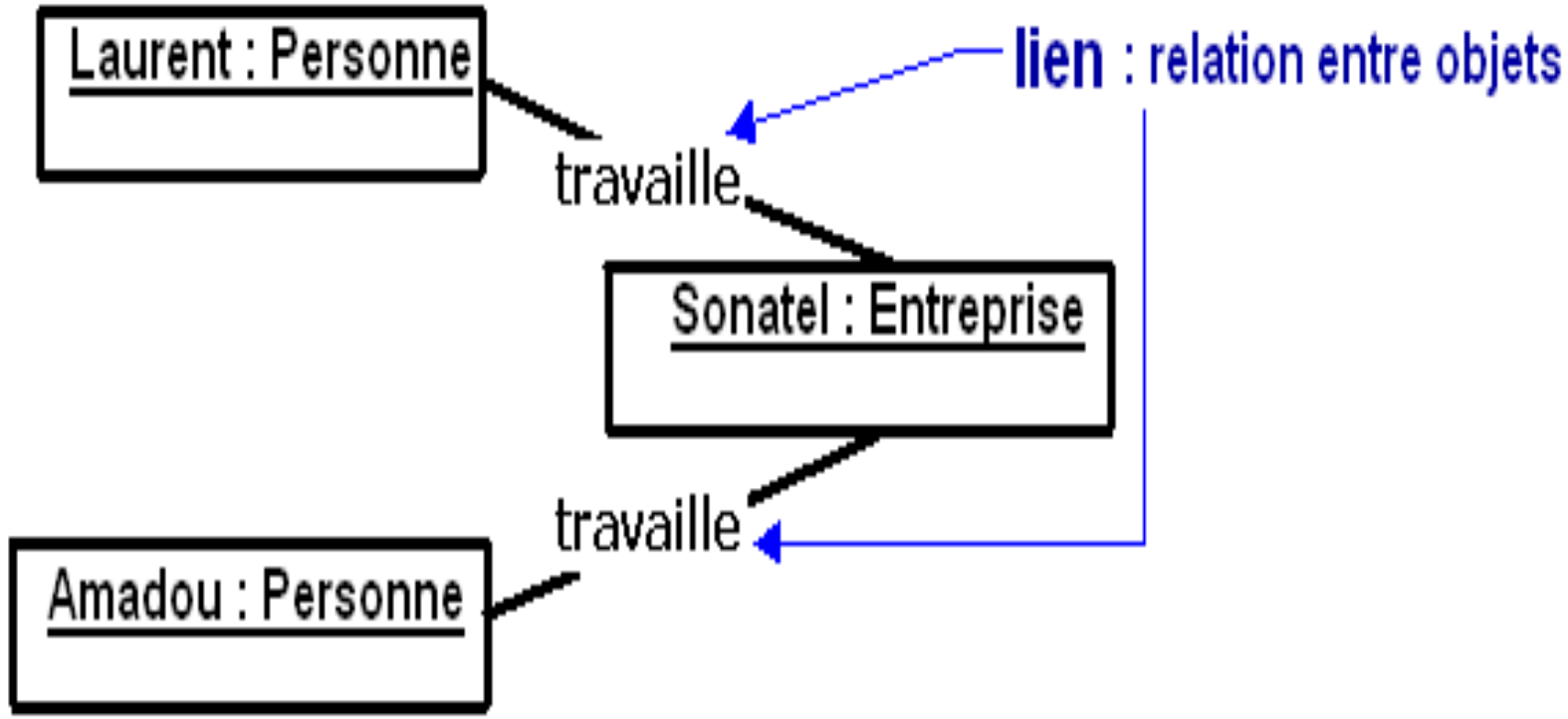
Une association exprime une connexion sémantique bidirectionnelle entre deux classes.

L'association est instanciable, sous forme de liens entre objets issus de classes associées.

Exemples d'associations de classes



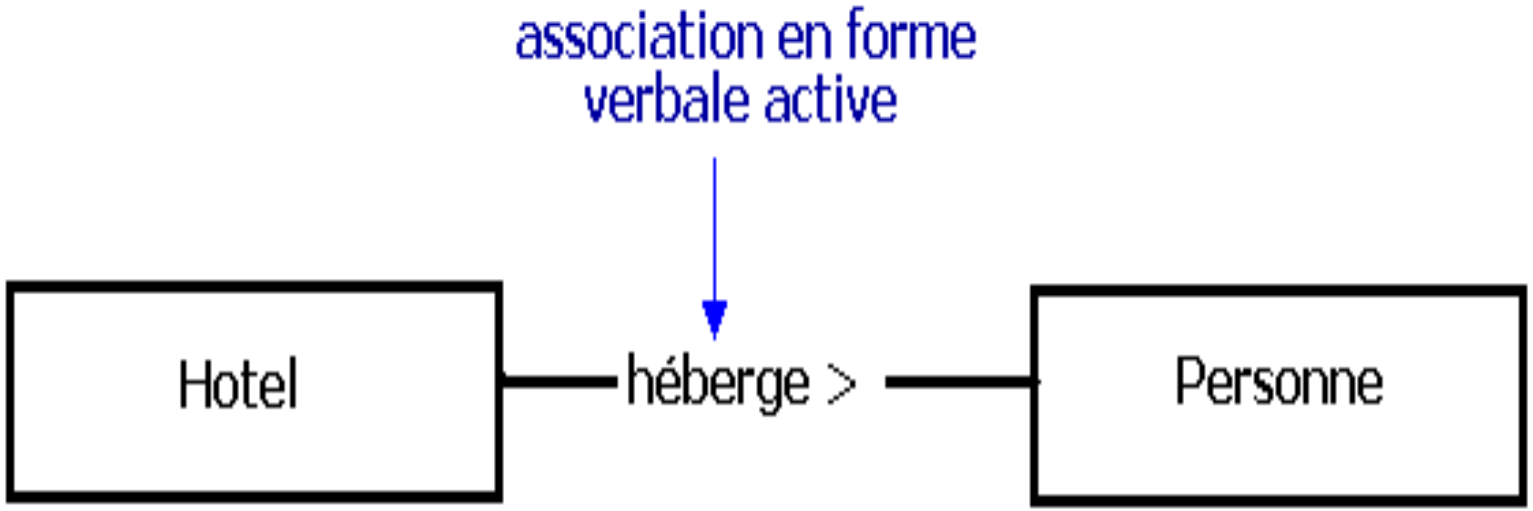
Exemples liens entre objets



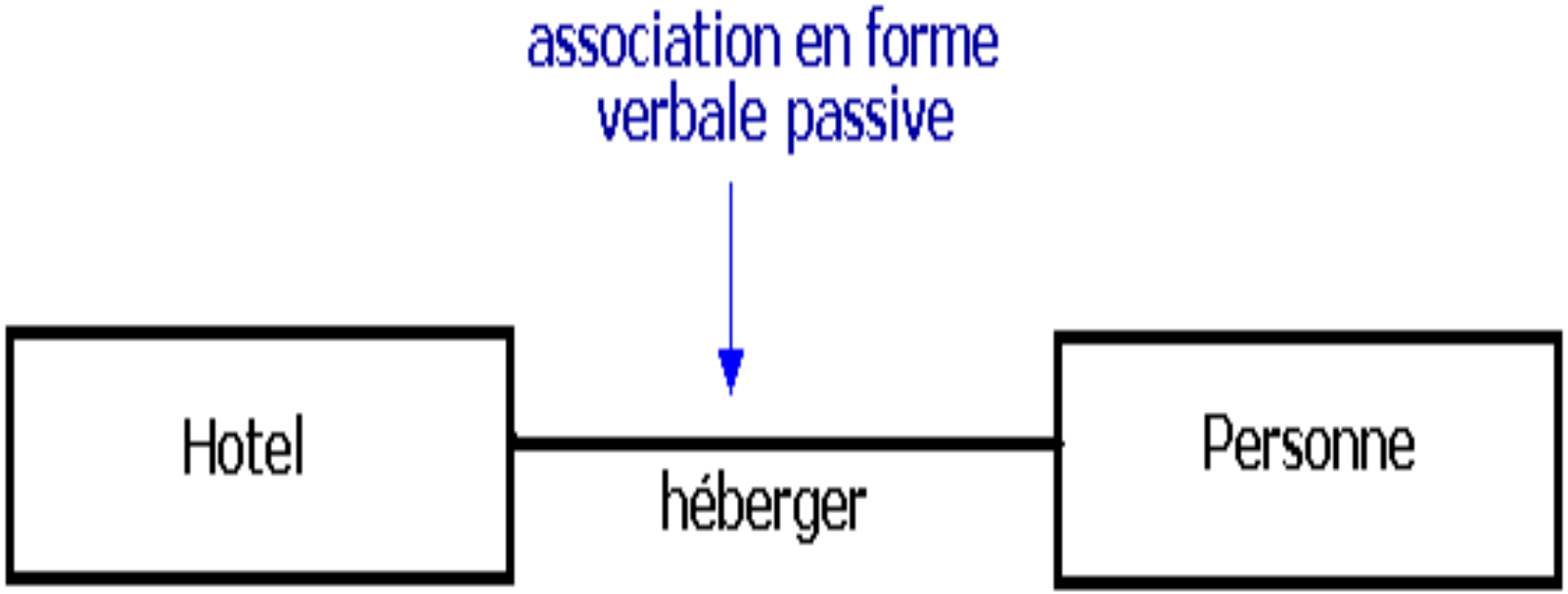
Association en forme verbale active

Précise le sens de lecture principal d'une association.

Exemple:



Association en forme verbale passive



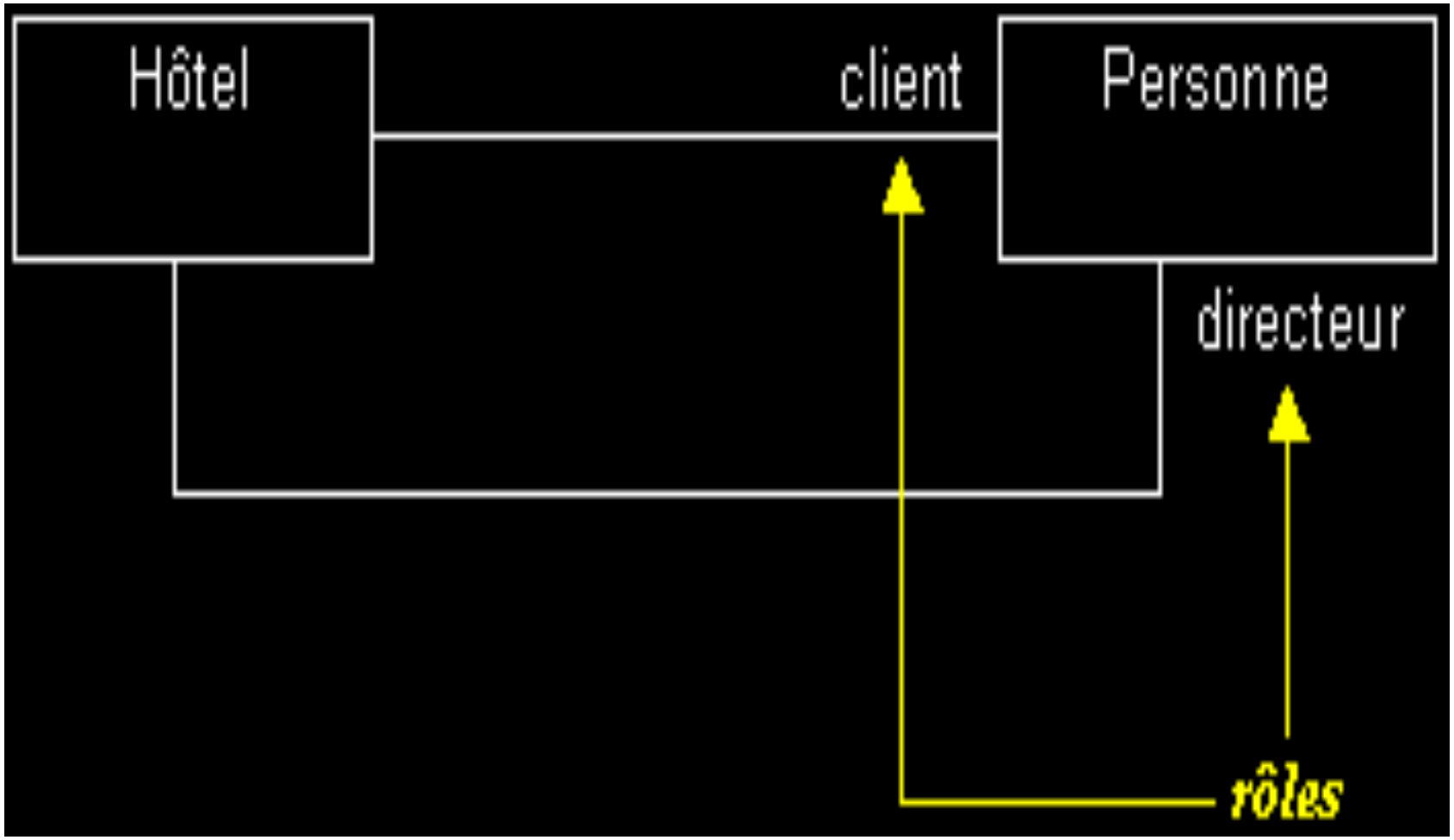
Lecture bidirectionnelle

Les Rôles dans une association

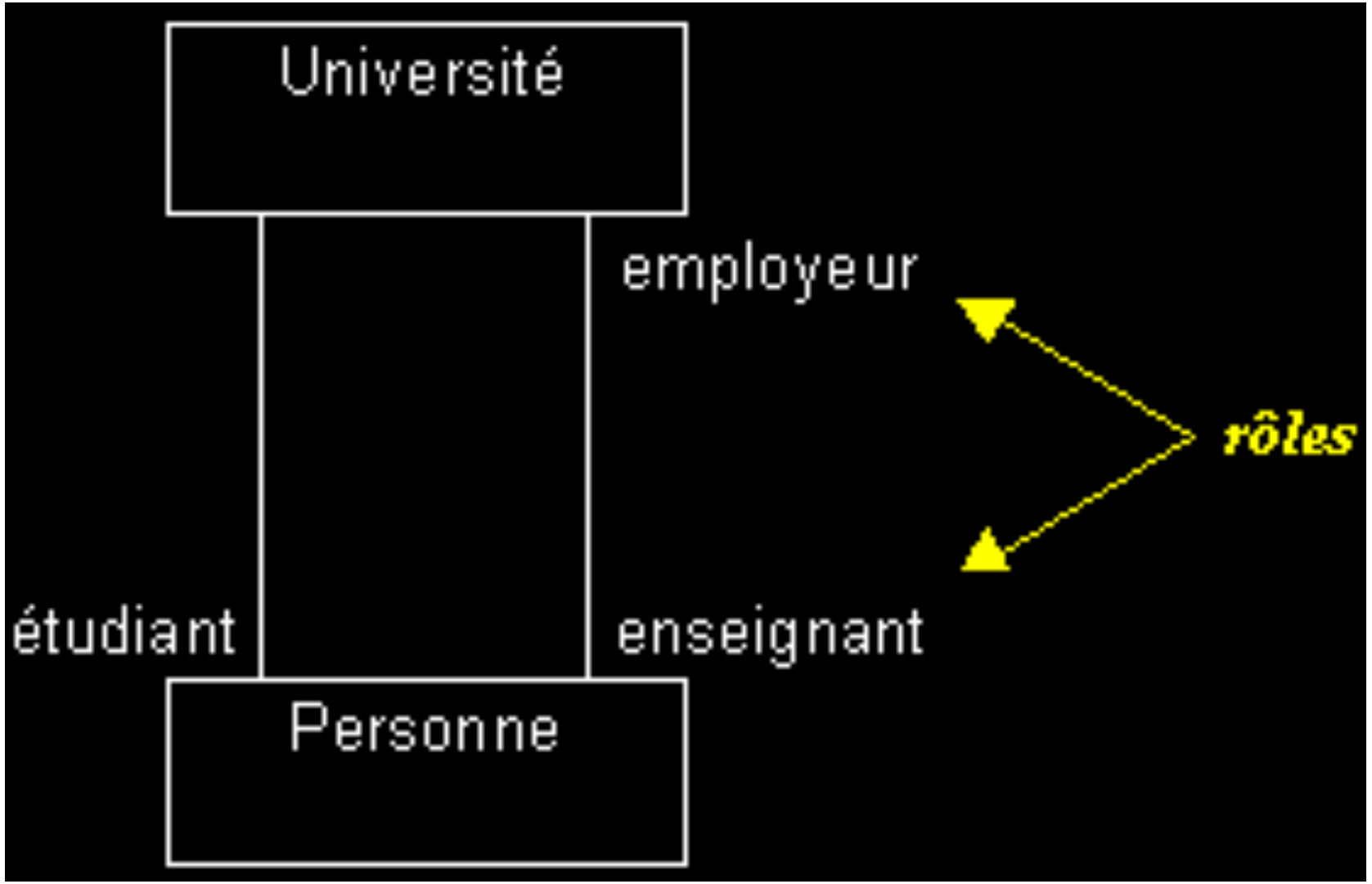
Un rôle spécifie une fonction d'une classe dans une association donnée (indispensable pour les associations réflexives).

Exemples de rôles

Exemple 1:



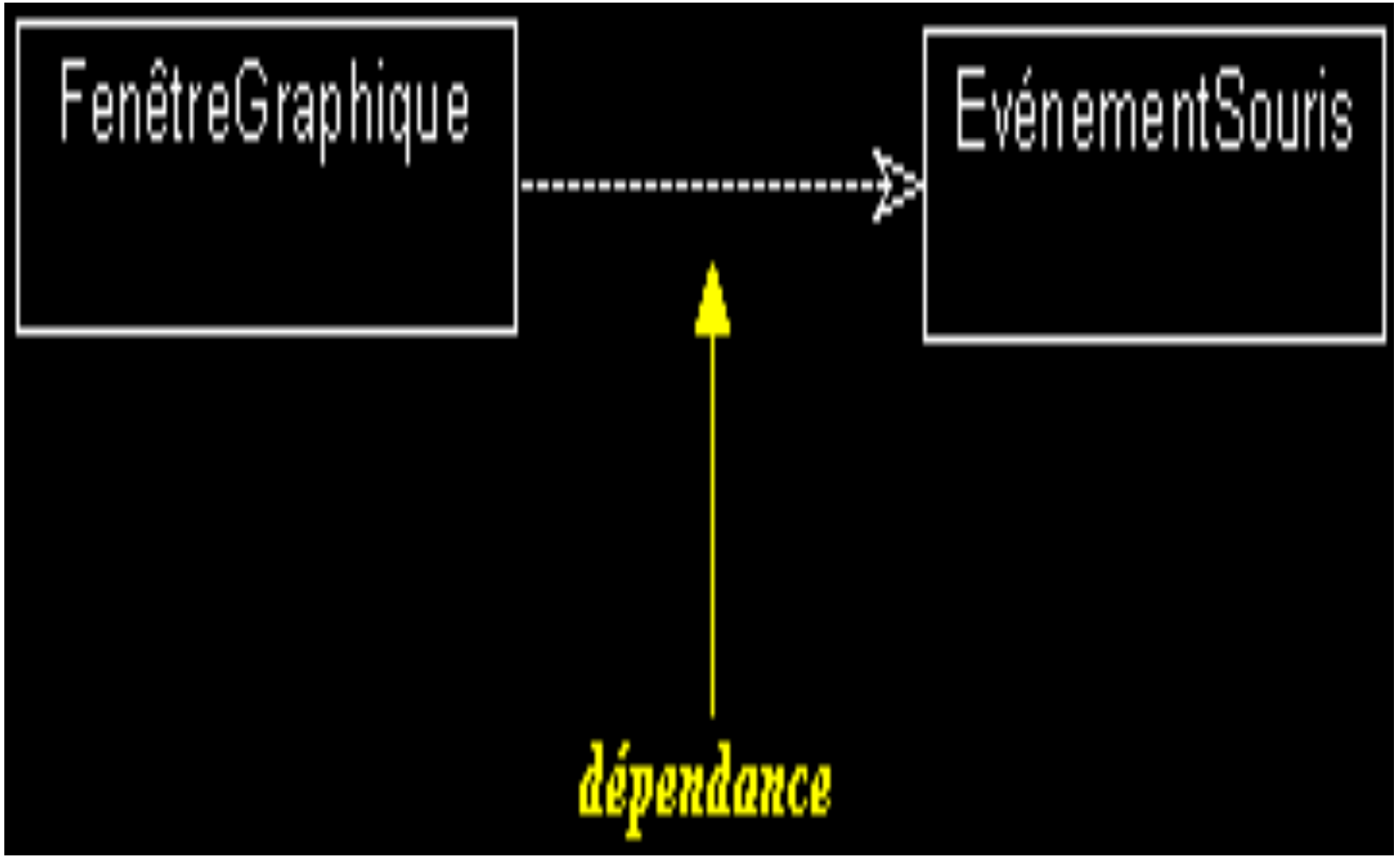
Exemple 2


Université
employeur
rôles
étudiant
enseignant
Personne

Relation de dépendance

- **Il s'agit d'une relation:**
  - **unidirectionnelle (qui n'est significative que dans un seul le sens; celui indiqué par une flèche);**
  - **et d'obsolescence (c-à-d si une classe B dépend d'une classe A; toute modification d'une classe A peut impacter la classe B).**

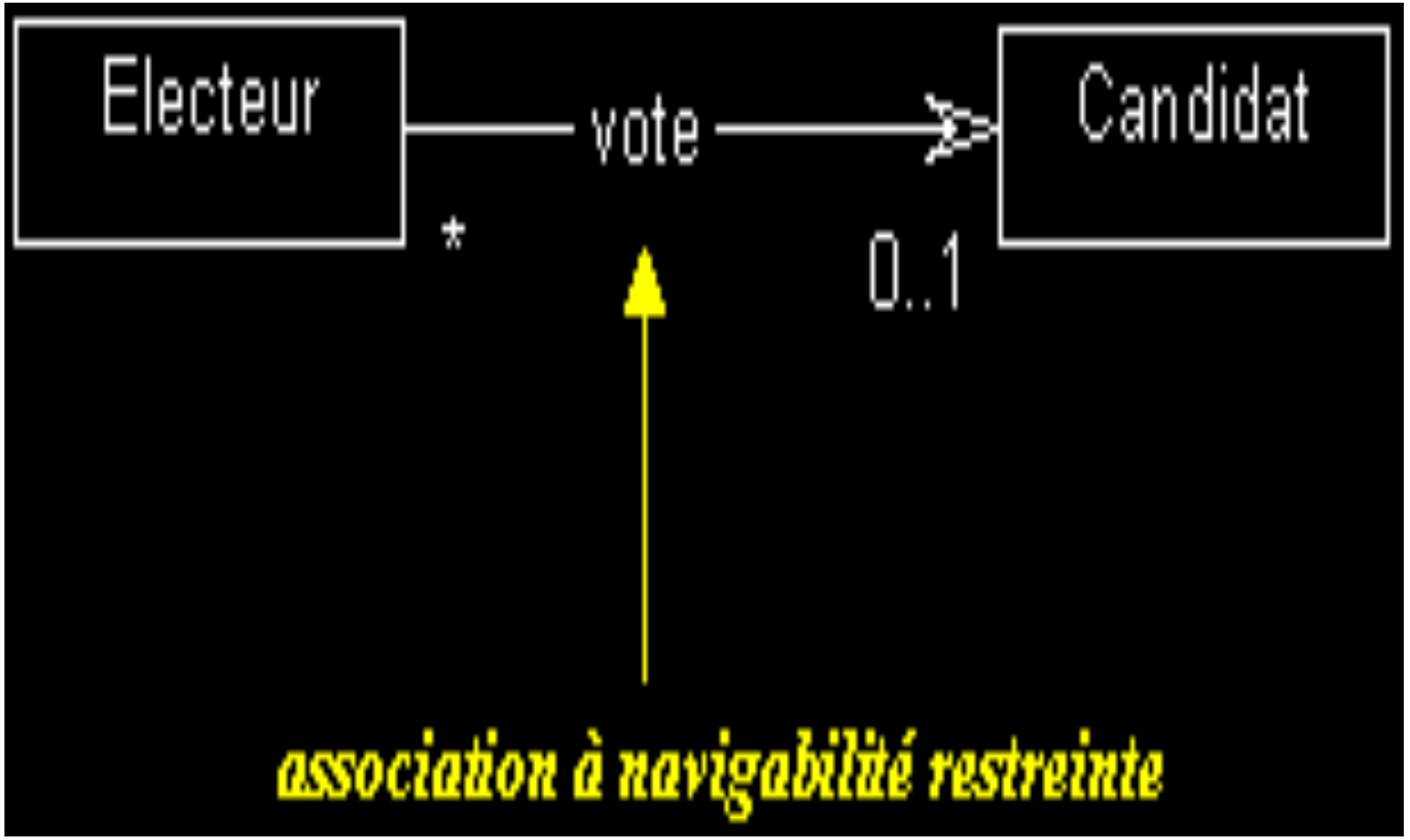
Exemple de Relation de dépendance



Association à navigabilité restreinte

- Par défaut, une association est navigable dans les deux sens. La réduction de la portée de l'association peut être réalisée en phase d'implémentation. Elle peut aussi être exprimée dans un modèle pour indiquer que les instances d'une classe ne "connaissent" pas les instances d'une autre classe .

Exemple d'association de navigabilité

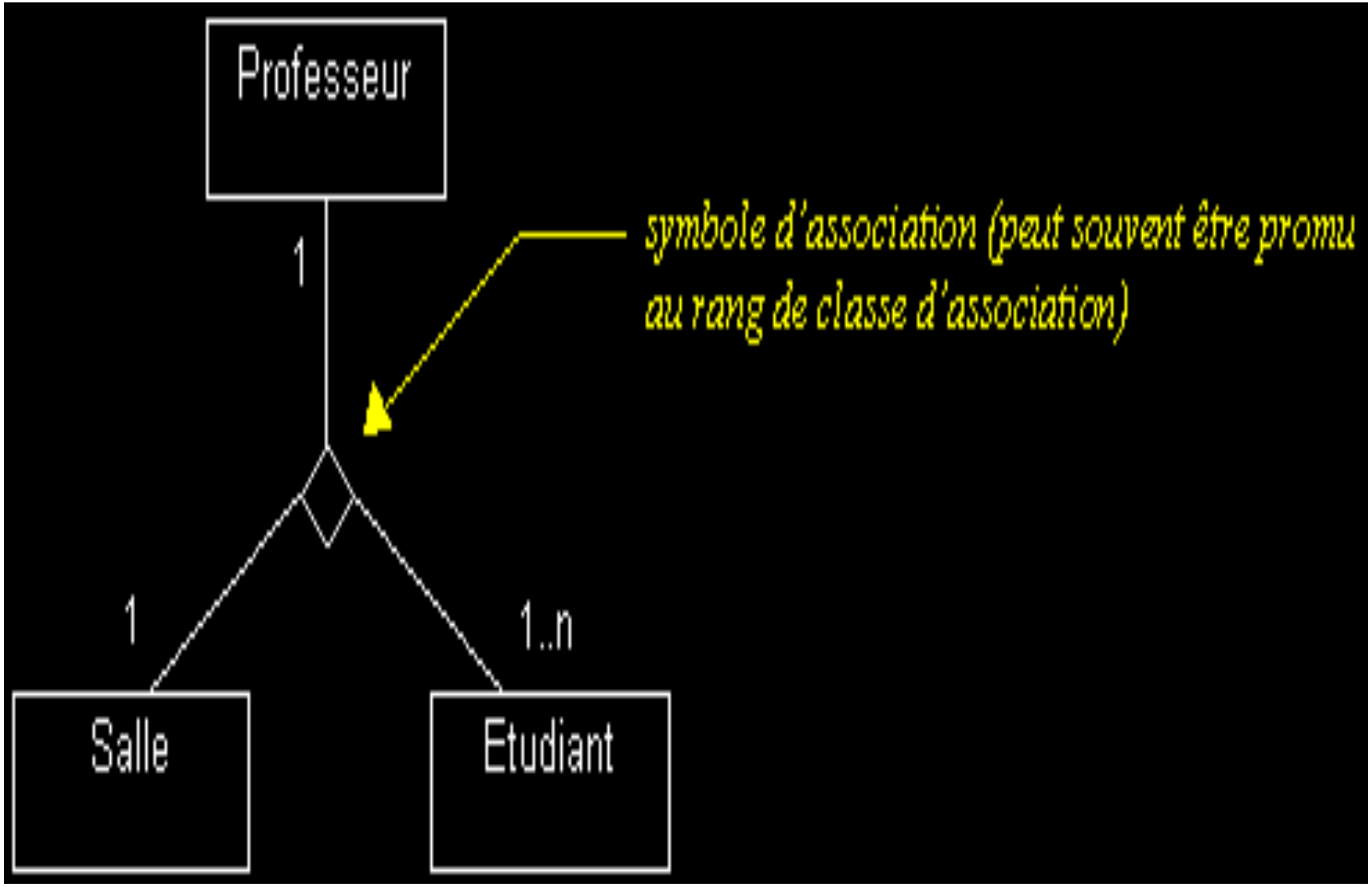


Association n-aire

Il s'agit d'une association qui relie plus de deux classes...

NB: De telles associations sont difficiles à déchiffrer et peuvent induire en erreur. Il vaut mieux limiter leur utilisation, en revenant aux associations binaires ou réflexives.

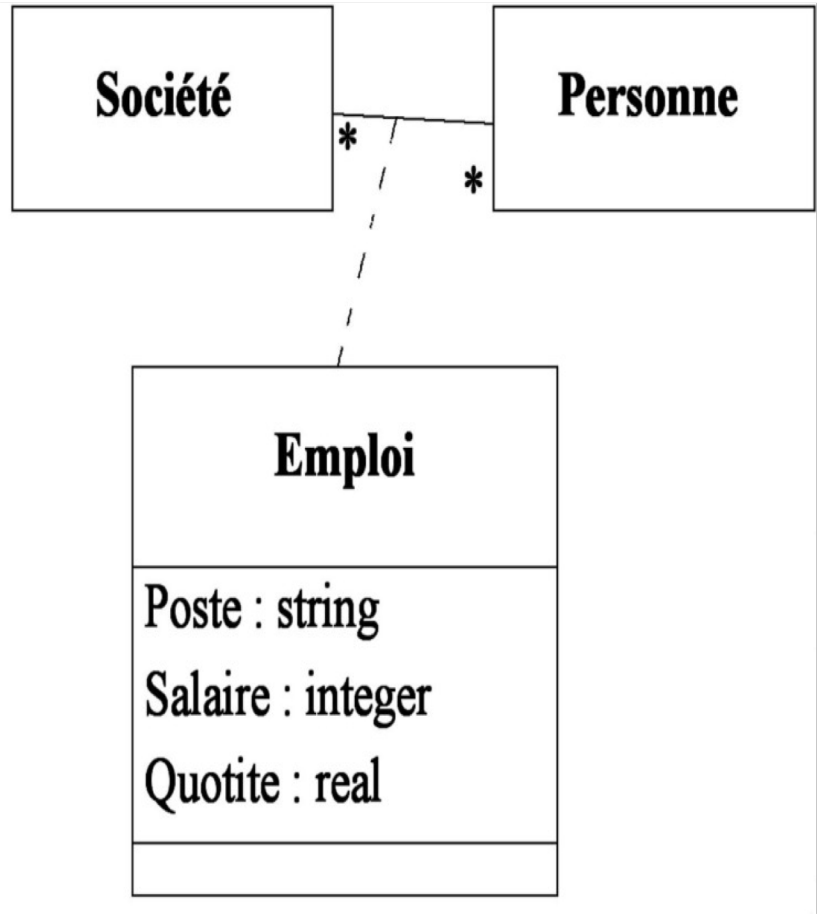
Exemple d'Association n-aire



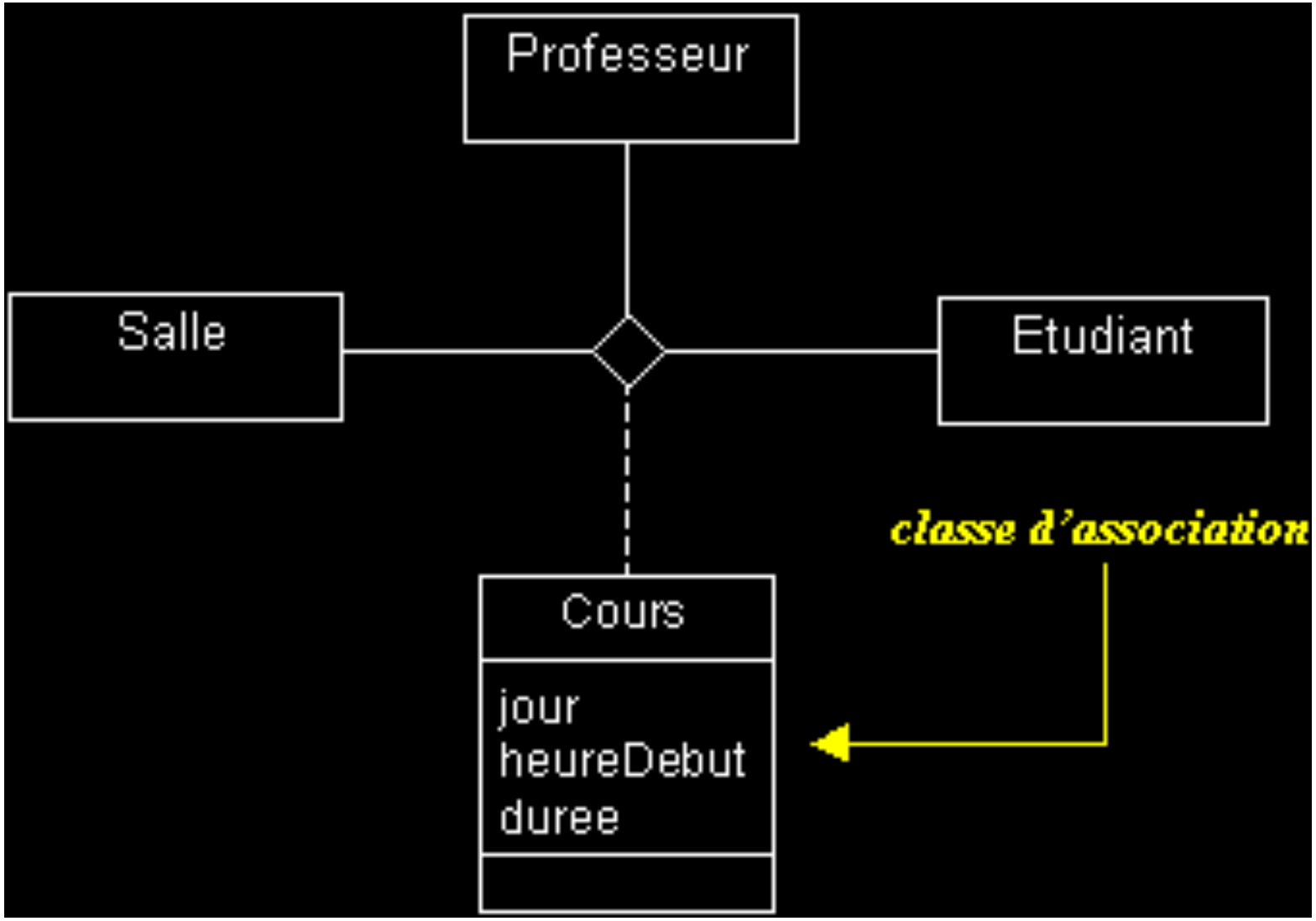
Classe d'association

Il s'agit d'une classe qui réalise la navigation (ie la relation) entre les instances d'autres classes.

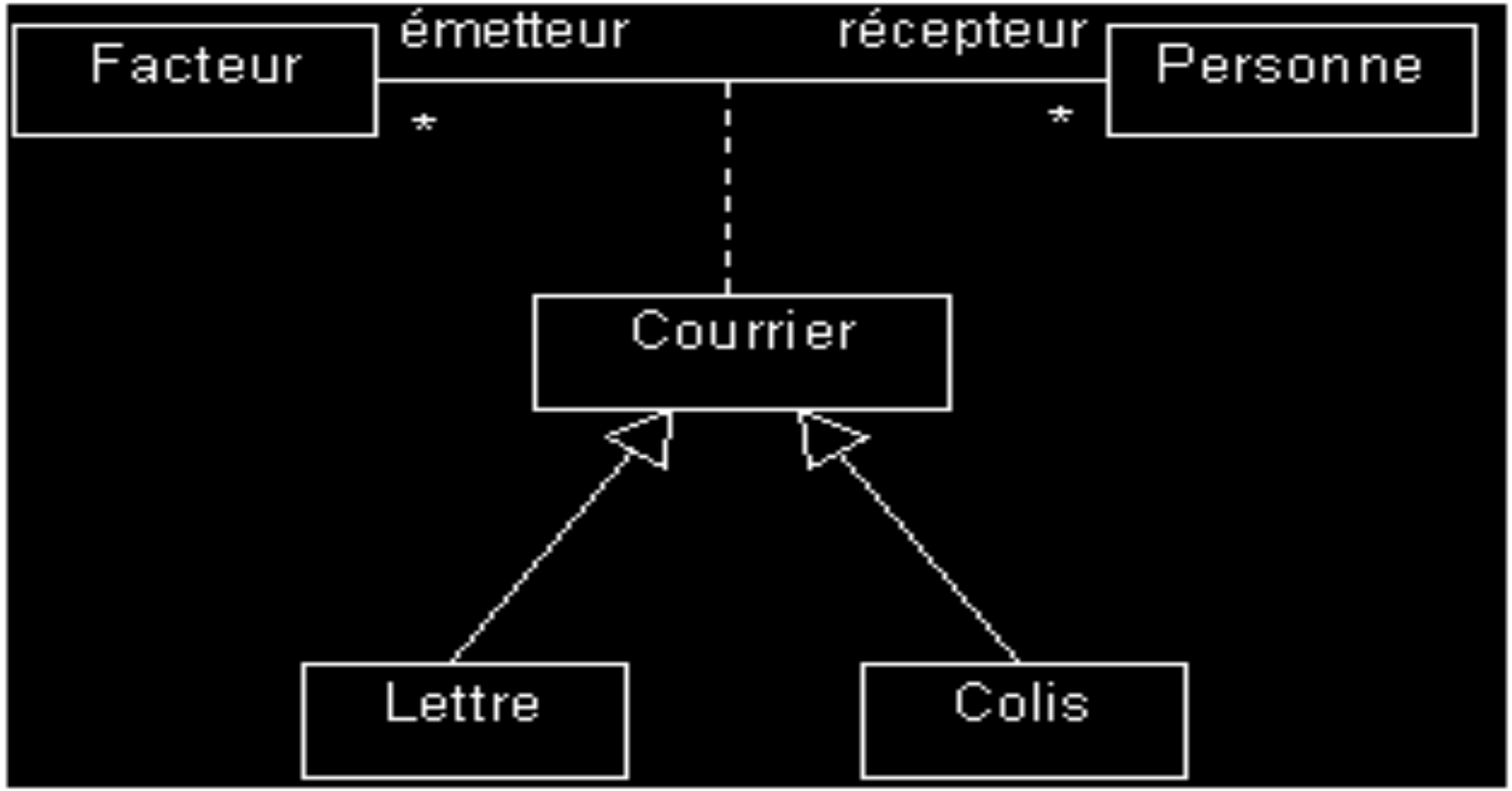
Exemple 1 de classe d'association



Exemple 2 de classe d'association



Exemple 3 de classe d'association

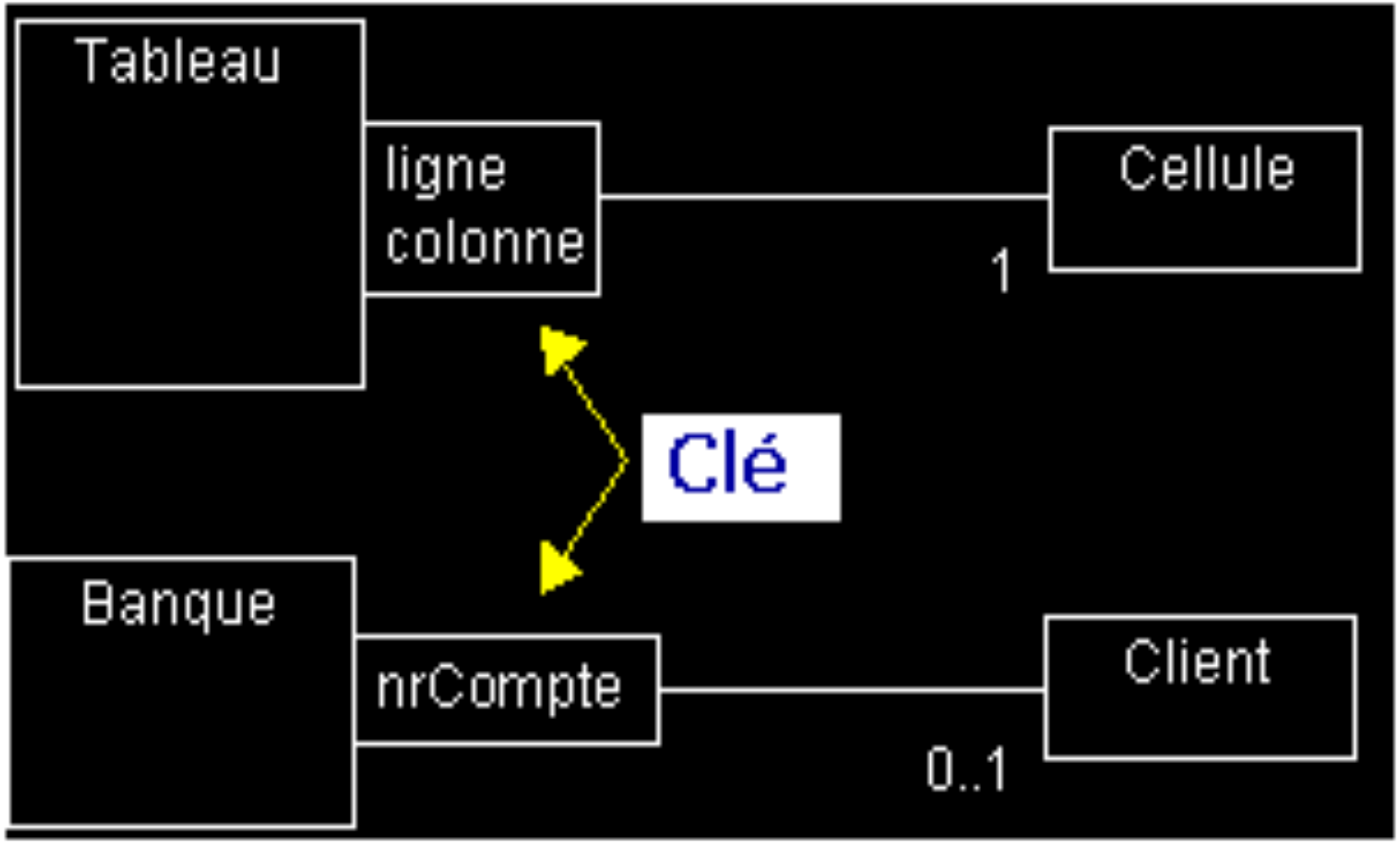


Classe d'association Courrier
existe pour des raisons d'héritage

Qualification

Permet de sélectionner un sous-ensemble d'objet(s) (qui peut être réduit à un singleton), parmi l'ensemble des objets qui participent à une association.
La restriction de l'association est définie par une clé, qui permet de sélectionner le ou les objets ciblés.

Exemple 1 de Qualification

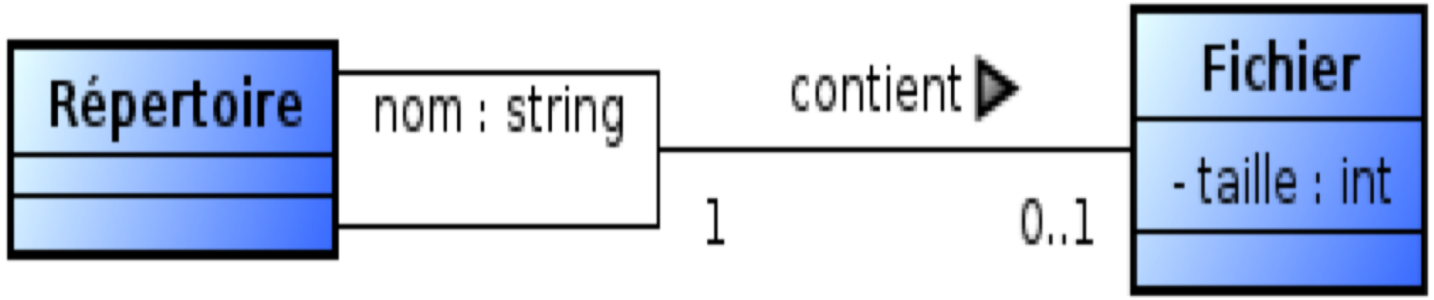


Exemple 2 de Qualification

Sans qualification



Avec qualification



Contrainte sur une association

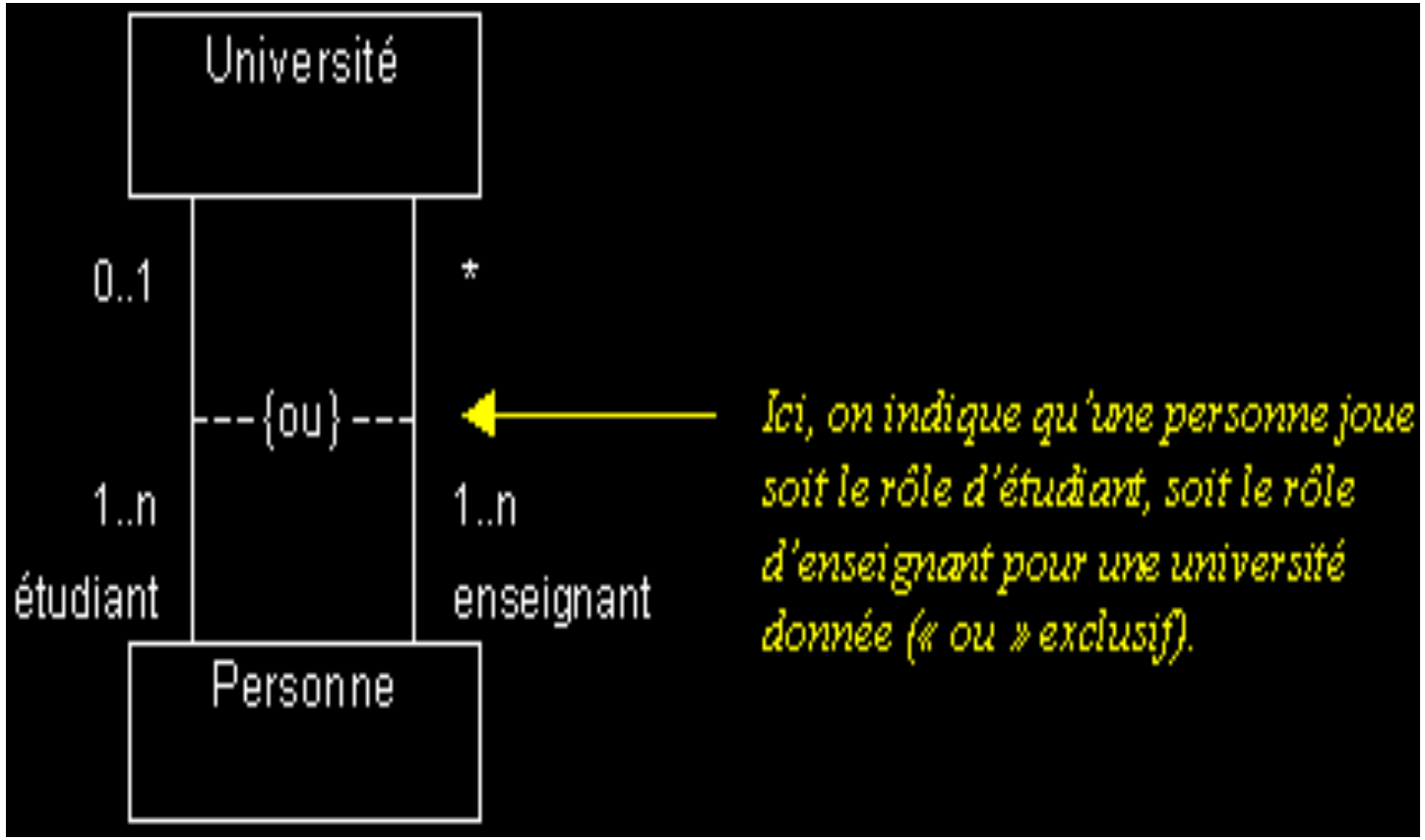
- Les contraintes sont des expressions qui précisent le rôle ou la portée d'un élément de modélisation (elles permettent d'étendre ou préciser sa sémantique).

- Sur une association, elles peuvent par exemple restreindre le nombre d'instances visées (ce sont alors des "expressions de navigation », cela correspond alors à une qualification).

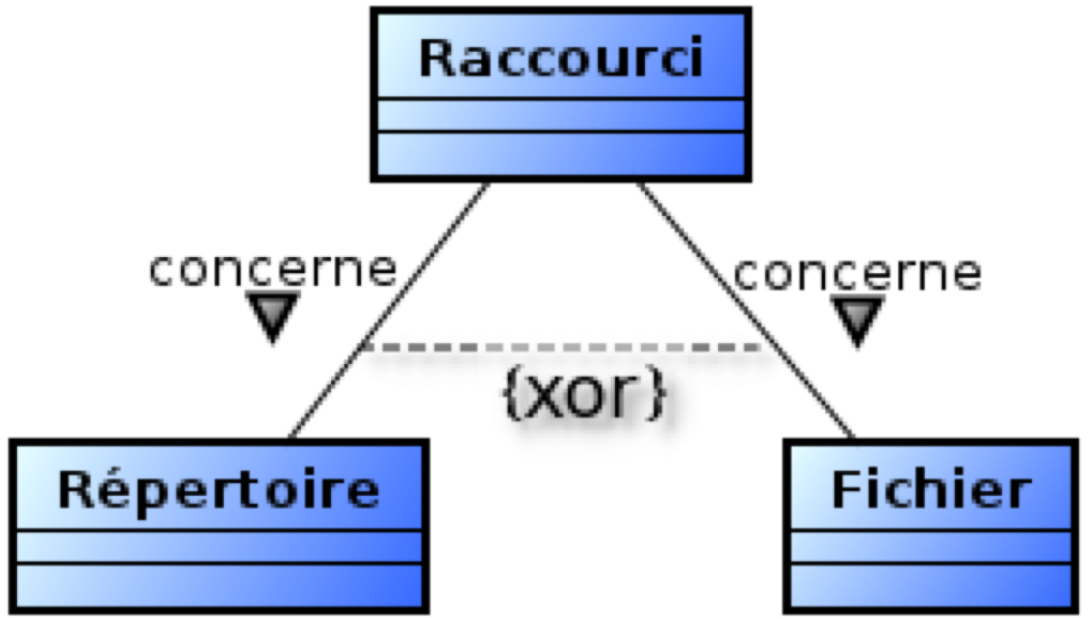
- Les contraintes peuvent s'exprimer en langage naturel. Graphiquement, il s'agit d'une expression encadrée d'accolades.

- NB: La qualification est un cas particulier de contrainte.

Exemple1 de contrainte:
Relation d'exclusion: {OU} EXCLUSIF (XOR)



Exemple 2 de contrainte:
Relation d'exclusion: {OU} EXCLUSIF (XOR)



Exemple 2 de contrainte

Relation d'inclusion: {subset} (sous-ensemble)

- Exprime que les objets qui participent à une association donnée constituent un sous-ensemble d'autres objets qui participent à une autre association.

- **Exemple**: l'association « **arbitrer** » entre **Match** et **Arbitre,** les :**Arbitres** font parti des :**Spectateurs** de la relation « **suivre** » entre **Match** et **Acteur.**

Exemple 3 de contrainte

Relation d'ordre: {ordered} (objets ordonnés)

- Cela signifie que les **objets** de la **classe concernée** sont ordonnés suivant les valeurs d'un des attributs de la classe;

- **Exemple**: Dans la relation « ***Appartenir*** » entre **Famille** et **Enfant** si on souhaite que la liste des enfants soit ordonnée selon l'attribut ***age***, on pré-positionne le mot clé **{ordered}** du côté de la classe **Enfant.**

Exemple 4 de contrainte:
Contrainte d'interdiction de modification de la nature d'une association {frozen}

Contraintes sur les associations
contrainte {frozen}

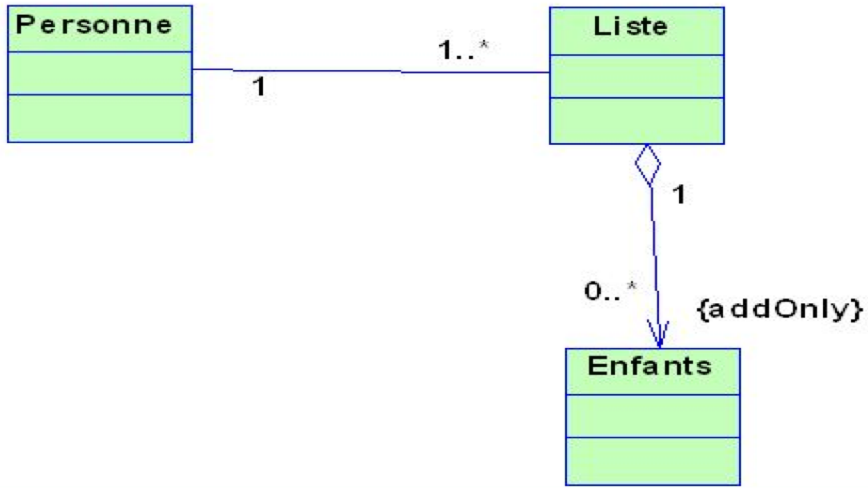
La contrainte prédéfinie {frozen} interdit l'ajout, la suppression ou la mise à jour des liens d'un objet vers les objets de la classe associée, après l'initialisation du premier.



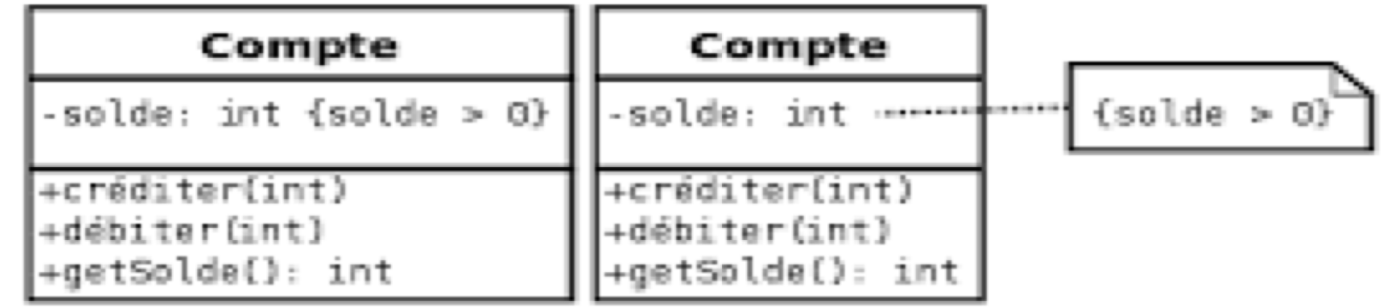
Exemple 5 de contrainte:
Contrainte d'autorisation d'ajout uniquement et non de suppression {addOnly}

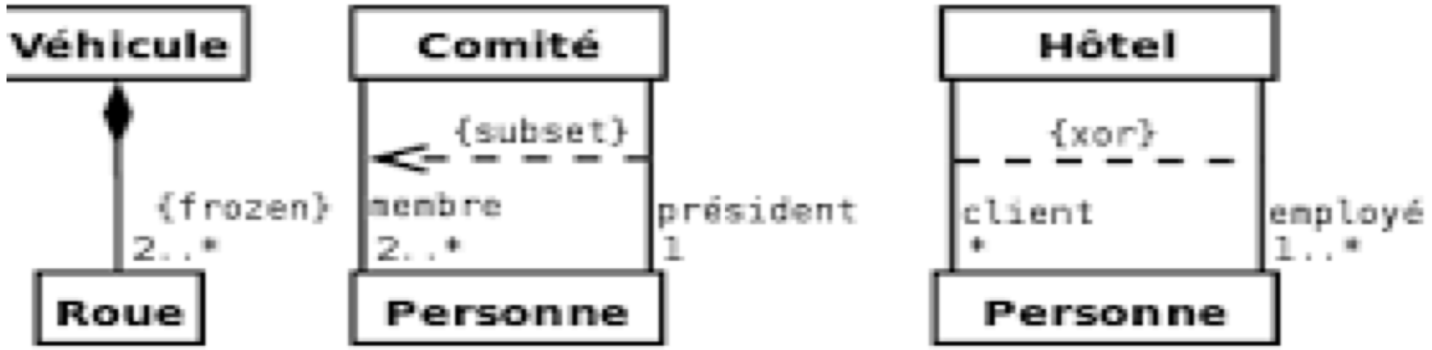
Contraintes sur les associations
contrainte {addOnly}

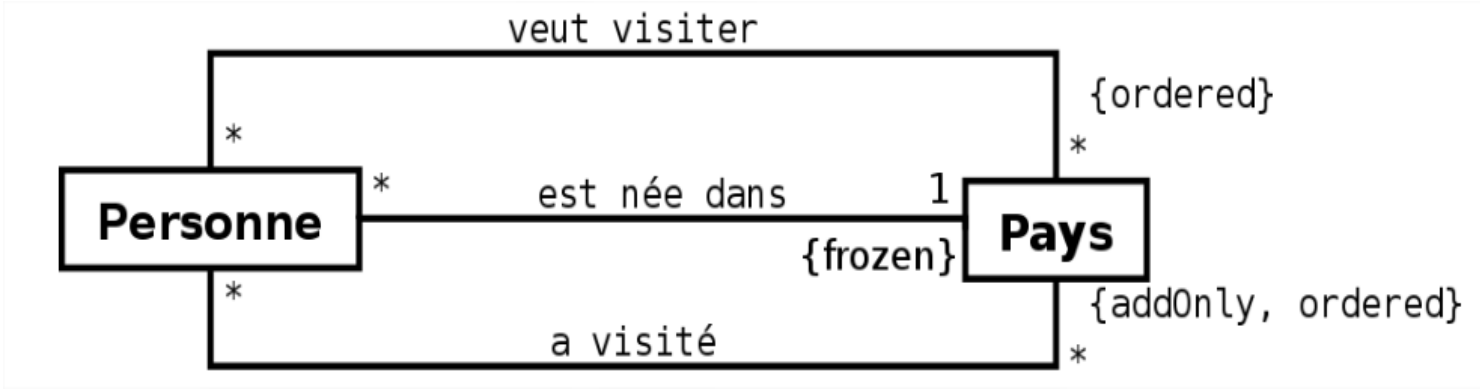
La contrainte prédéfinie {addOnly} autorise l'ajout de nouveaux objets, mais pas leur suppression ni leur mise à jour.



Autres exemples







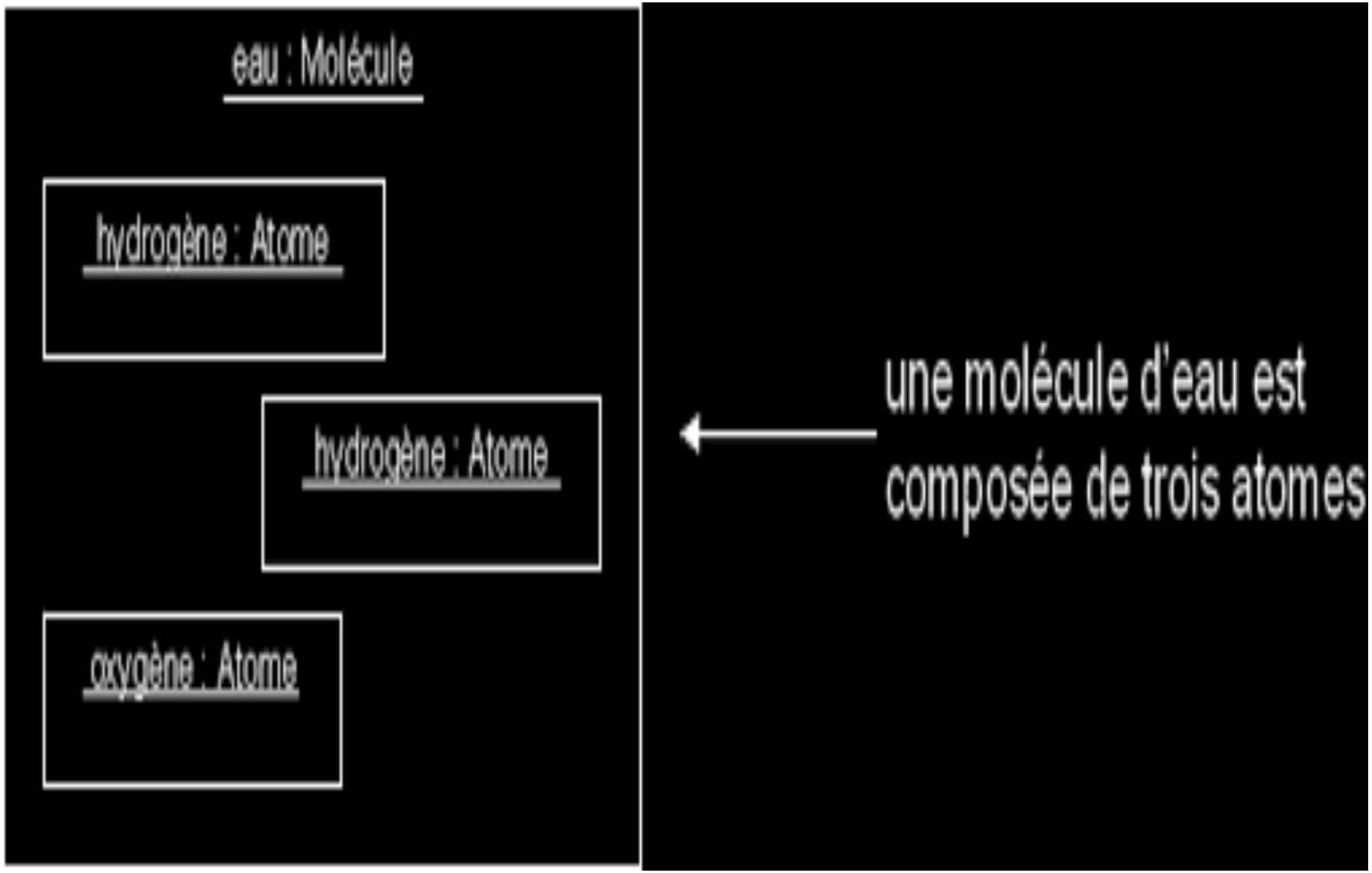
II-4-2 Agrégation

- **« L'agrégation** est une association non symétrique, qui **exprime un couplage fort** et **une relation de subordination ».**

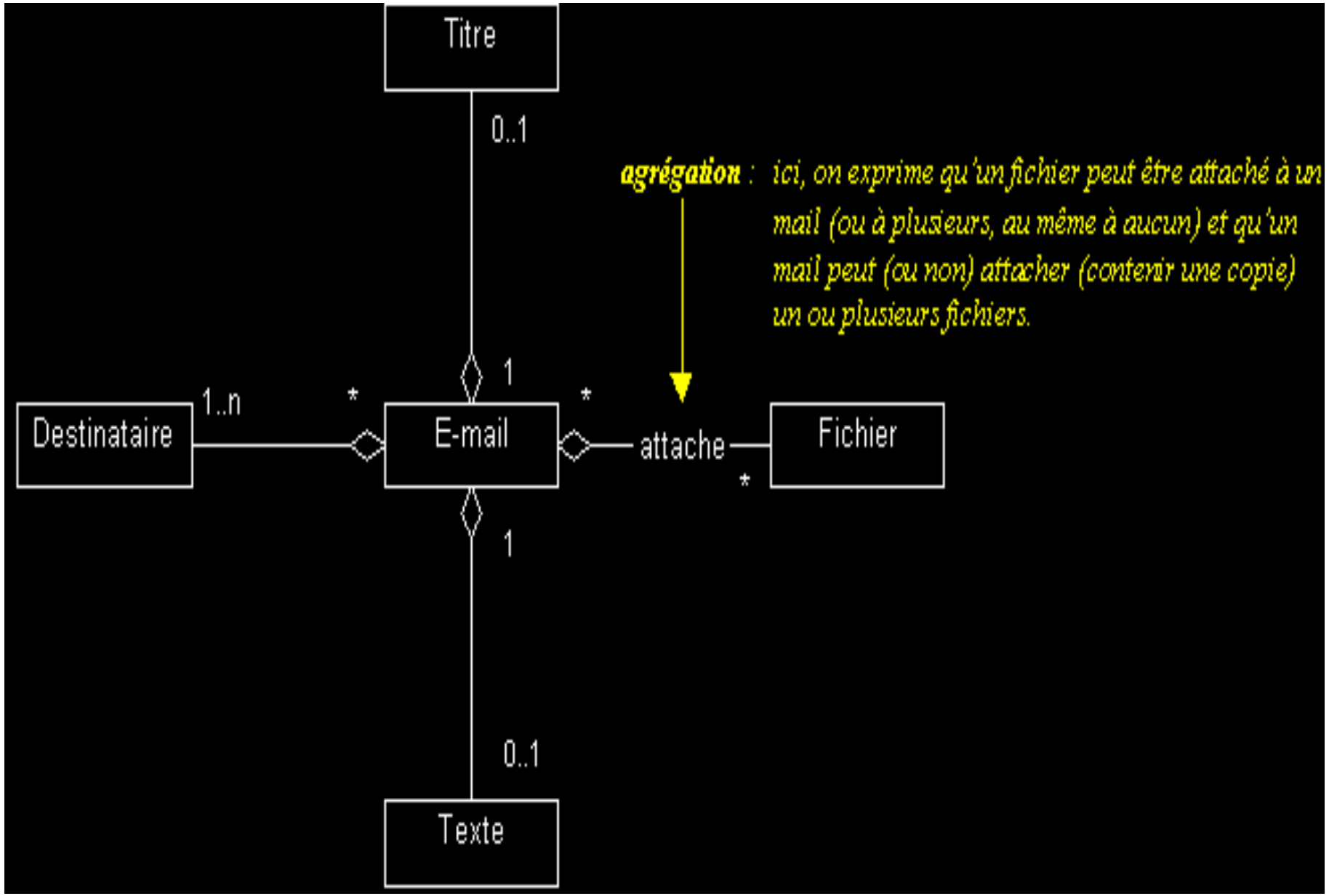
Elle représente une **relation** de type **"ensemble / élément »,**
« agrégat / élément agrégé » ou encore **« composé / composant »**

- **Un composant peut être lié à plusieurs composés.**
- **La suppression** (ou destruction) **d'un composé n'entraine pas nécessairement la suppression** de ces **composants;** ces derniers peuvent exister sans le **composé**;
- **Ainsi**
  - un **changement d'état** d'un **composant**, **peut entraîner** un **changement d'état** du (ou des) **composé(s)** associés;
  - Tout comme **le comportement d'un composé peut impacter celui des composés**;

Exemple1 d'agrégation



Exemple 2 d'agrégation



II-4-3 Composition

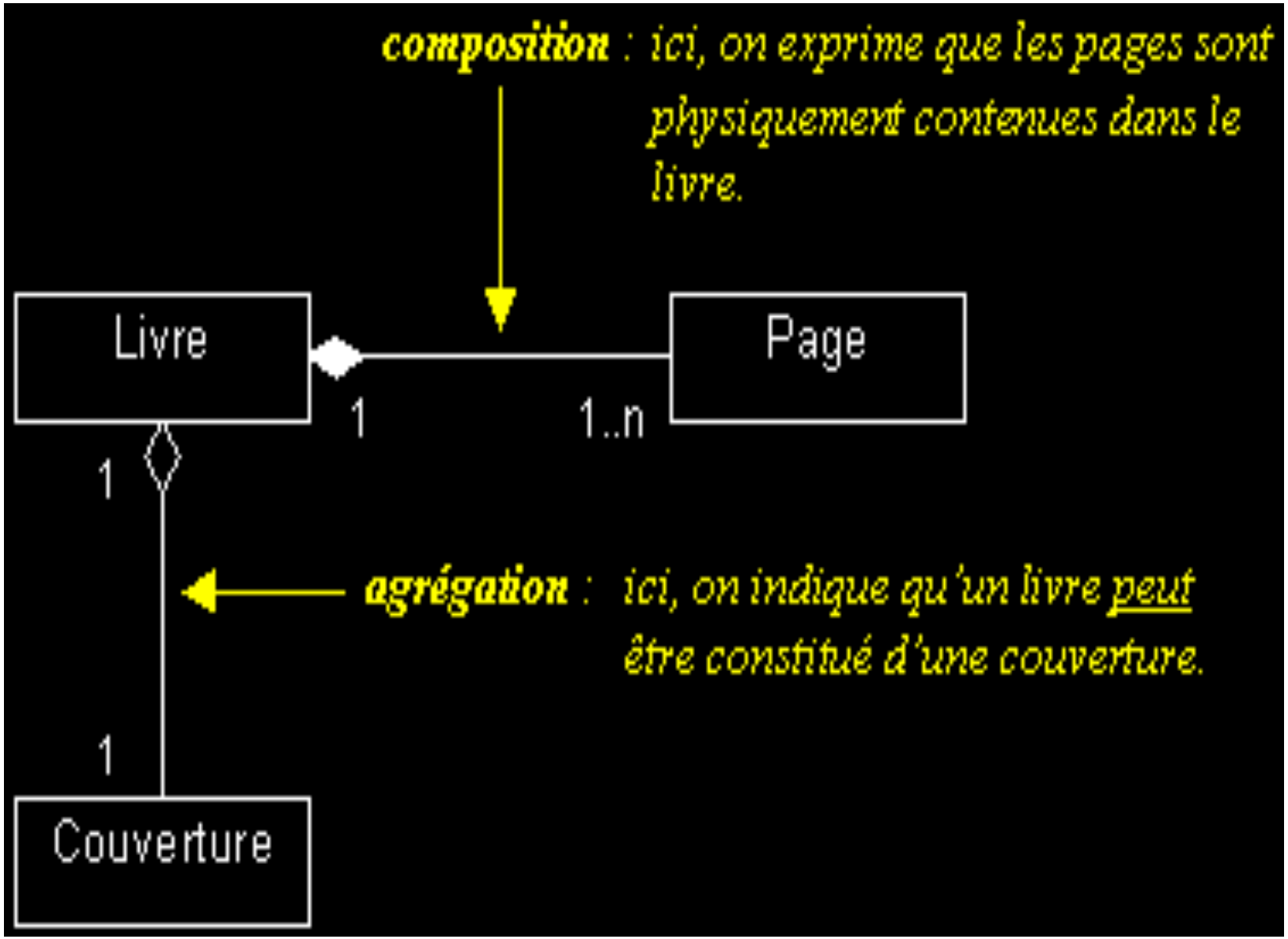
- **La composition est une agrégation forte: agrégation par valeur; le composé et les composants sont physiquement liés;**

- **Un composant** est **une partie non partagée d'un composé**; (unicité de la participation à un agrégat)

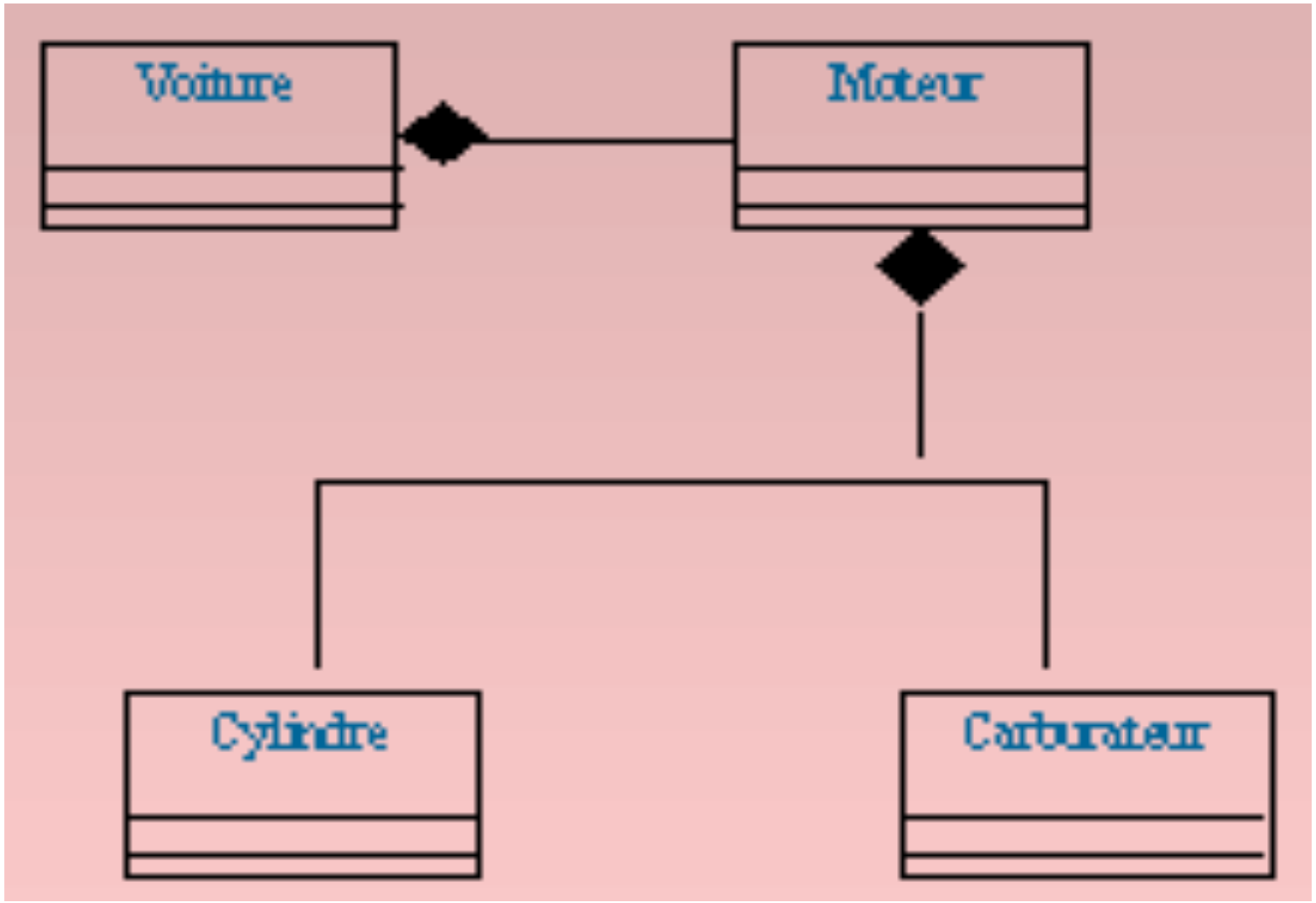
- En **Conséquence**: La **multiplicité** maximale, au niveau de l'objet composé, est obligatoirement de **1**;

- La **suppression** d'un objet **composé entraine** la **suppression** de ces **composants**.

Exemple de Composition



Autre exemple de Composition



Synthèse des différentes relations

- Association
  - Navigable dans les 2 sens : ——————
  - Navigable dans un sens : ——————>
    ou —X——————>
- Composition : ——————◆
- Agrégation : ——————◇
- Généralisation : ——————▷
- Réalisation d'une interface : ——————O ou - - - - - - - - - - - - ▷
- Utilisation d'une interface : ——————C ou - - - - «use» - - - - >
- Intanciation d'une classe générique : - - - - «bind» (Type) - - - - >
- Dépendance : - - - - - - - - - - - - >

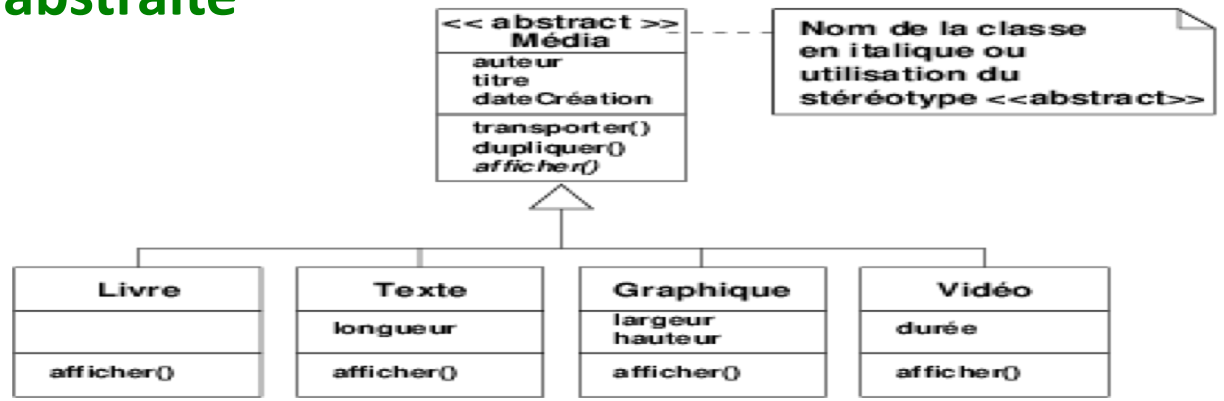
Réalisation – utilisation

Notations UML :

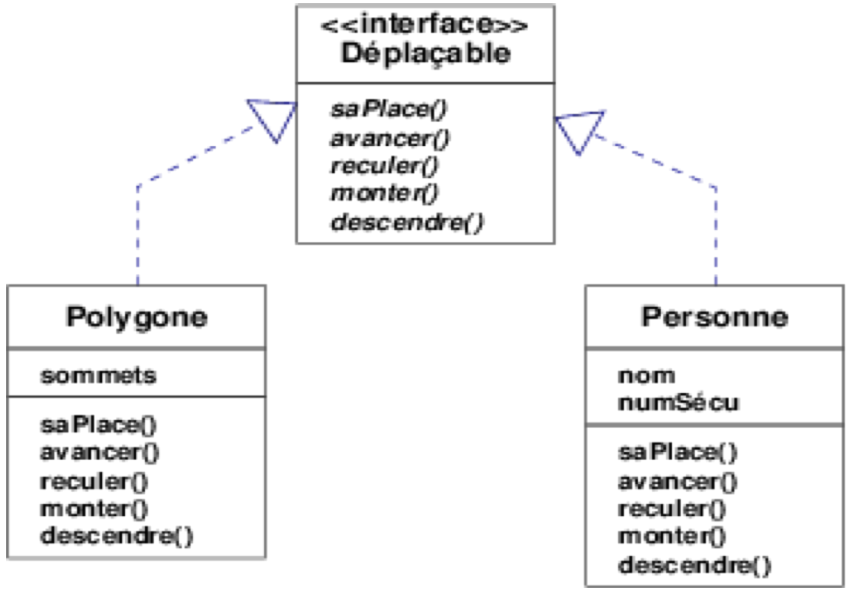
- Nom : «interface» Itf1 ou o
- Héritage : Itf1 —▷ Itf2 entre 2 interfaces
- Réalisation : Class1 - - - -▷ Itf1 ou Class1 —— Itf1 Selon les 2 représentations
- Utilisation : Class2 - - «use» - -> Itf1 ou Class2 —— C

Exemples 1

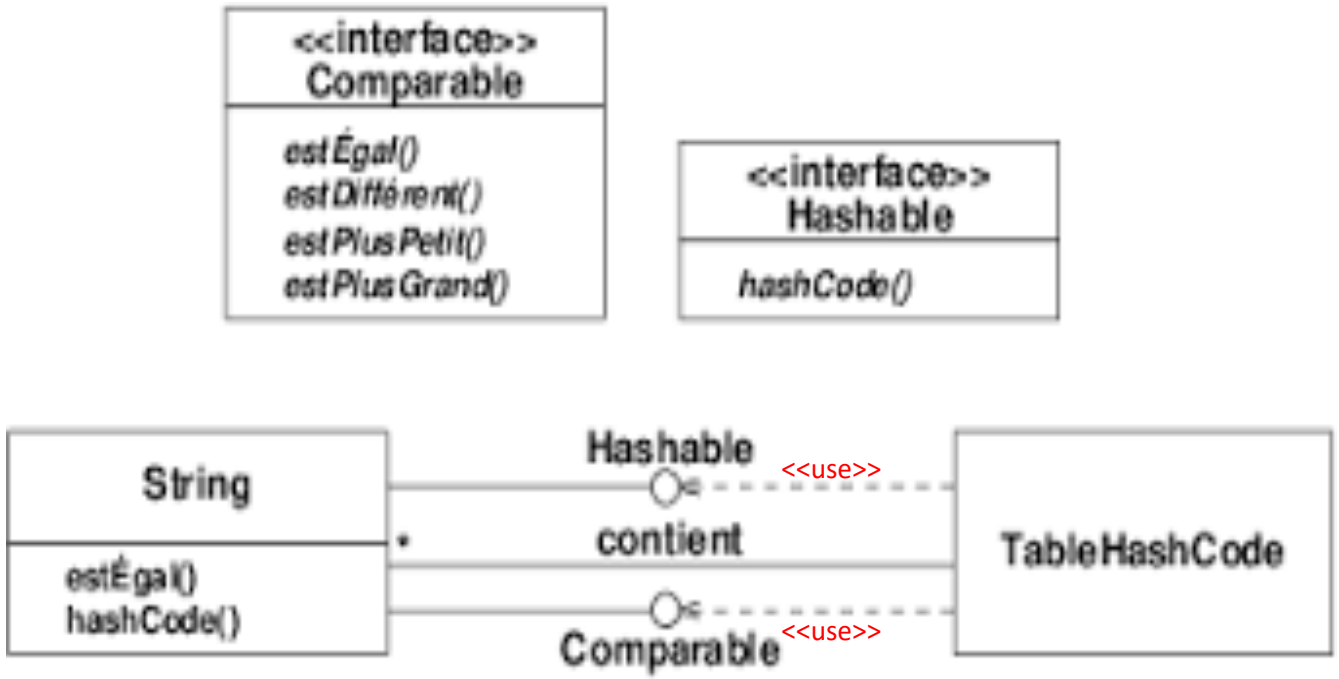
- **Classe abstraite**



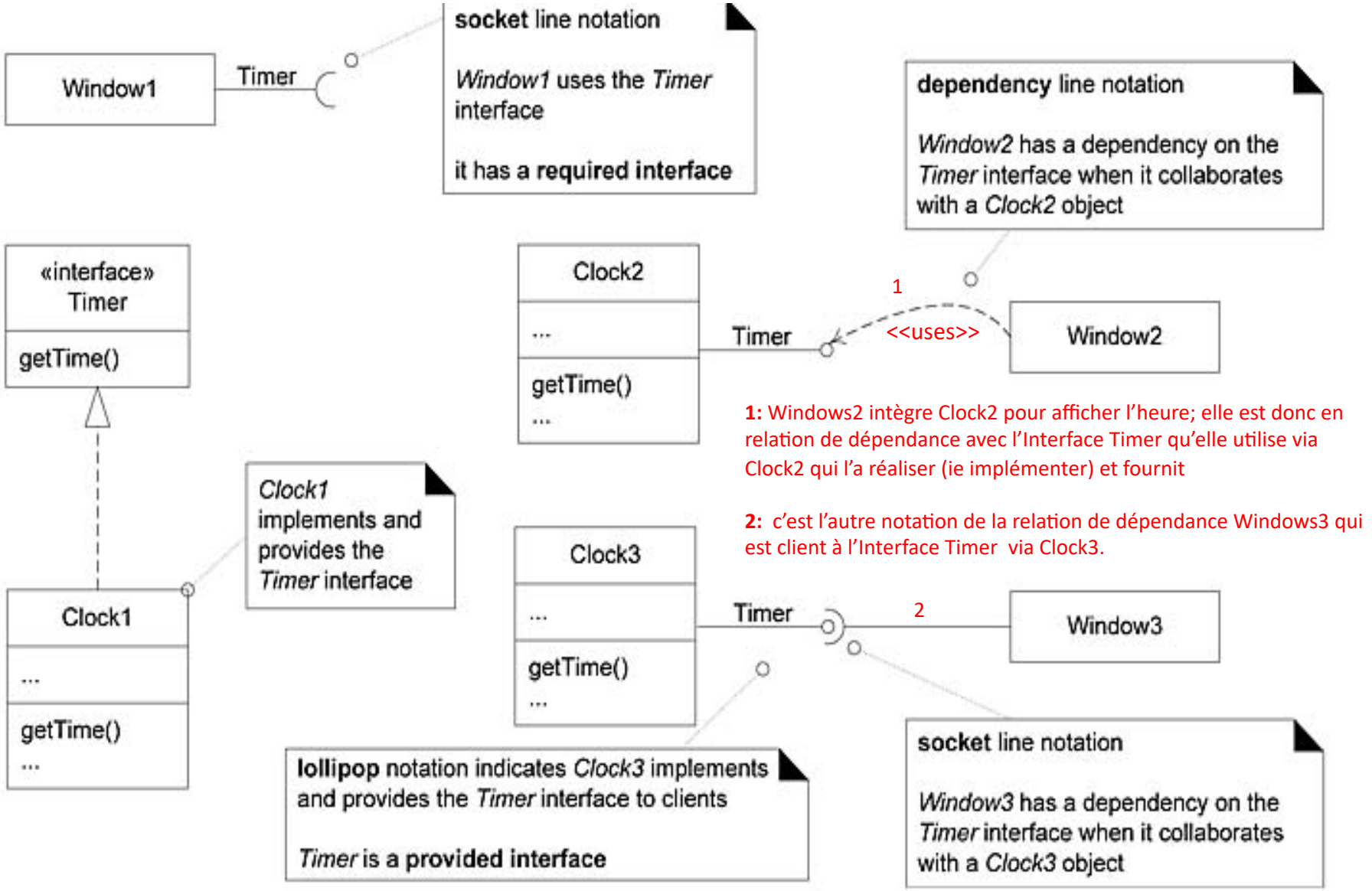
- **Interface**



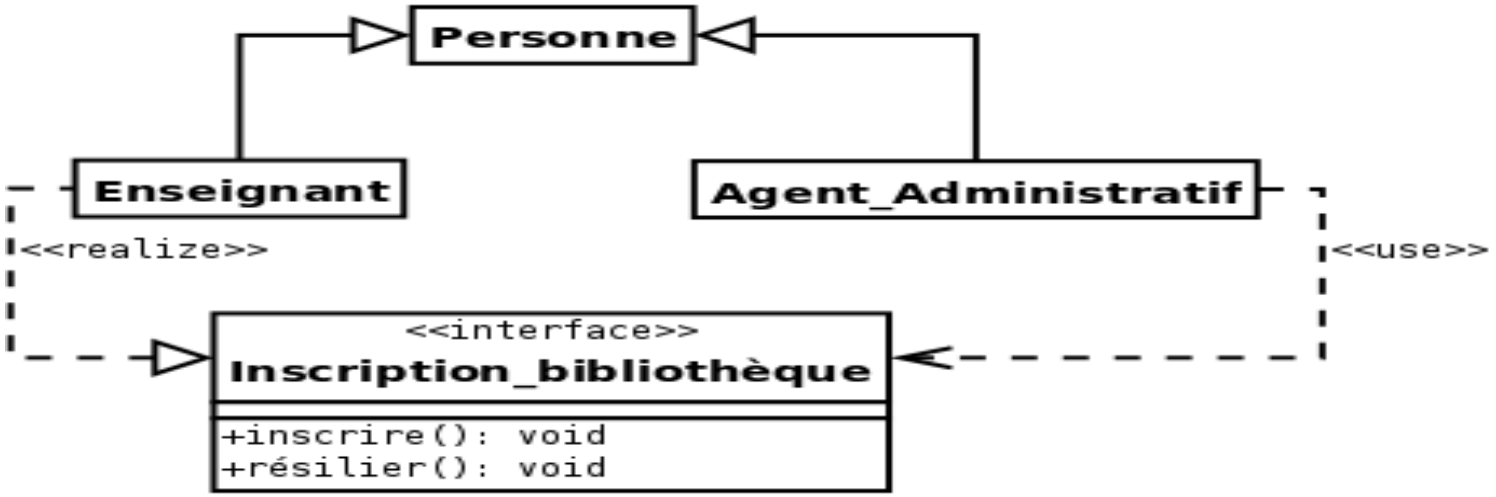
Exemples 2


«interface»
Comparable
estÉgal()
estDifférent()
estPlusPetit()
estPlusGrand()
«interface»
Hashable
hashCode()
String
estÉgal()
hashCode()
Hashable
<<use>>
contient
Comparable
<<use>>
TableHashCode

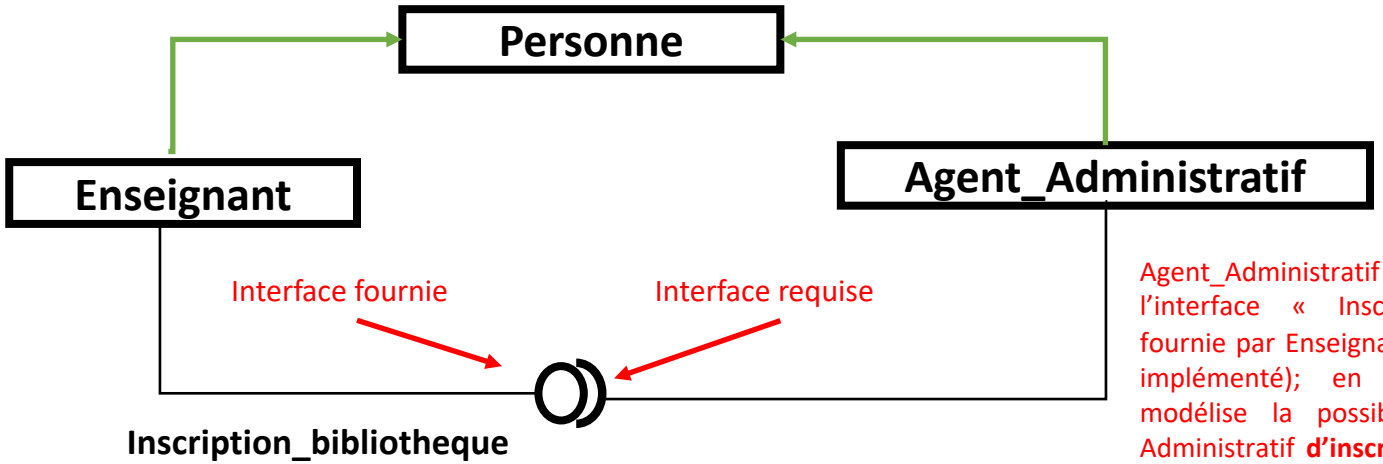
Exemples 3


Window1
Timer
socket line notation
Window1 uses the Timer interface
it has a required interface
dependency line notation
Window2 has a dependency on the Timer interface when it collaborates with a Clock2 object
«interface» Timer
getTime()
Clock2
...
getTime()
...
Timer
1
<<uses>>
Window2
1: Windows2 intègre Clock2 pour afficher l'heure; elle est donc en relation de dépendance avec l'Interface Timer qu'elle utilise via Clock2 qui l'a réaliser (ie implémenter) et fournit
2: c'est l'autre notation de la relation de dépendance Windows3 qui est client à l'Interface Timer via Clock3.
Clock1 implements and provides the Timer interface
Clock1
...
getTime()
...
Clock3
...
getTime()
...
Timer
2
Window3
lollipop notation indicates Clock3 implements and provides the Timer interface to clients
Timer is a provided interface
socket line notation
Window3 has a dependency on the Timer interface when it collaborates with a Clock3 object

Exemples 4:
de réalisation (realize) et de dépendance (uses) entre classe et interface



OU

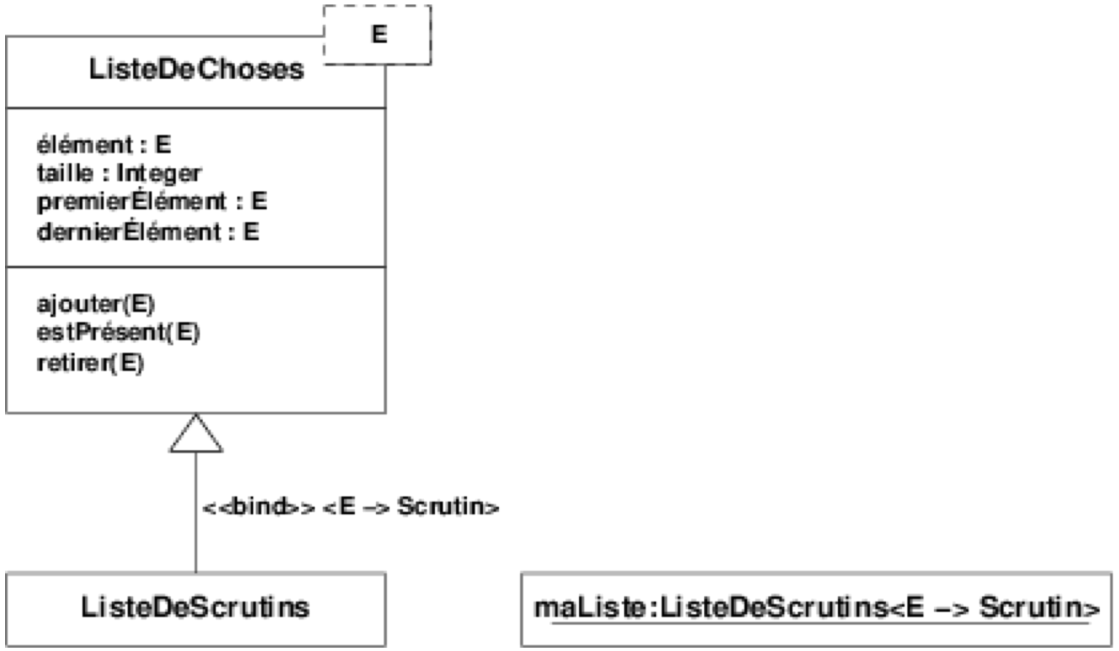


Agent_Administratif requiert c.-à-d. utilise l'interface « Inscription_bibilotheque » fournie par Enseignant qui l'a réalisé (c-à-d implémenté); en d'autres termes, on modélise la possibilité pour un Agent-Administratif **d'inscrire**() ou de **résilier**() un enseignant à la bibliothèque.

Implémentation d'un Template

- L'implémentation correspond à la **spécification** (c-à-d à la **précision**) des **paramètres** génériques;

- **Exemple**:



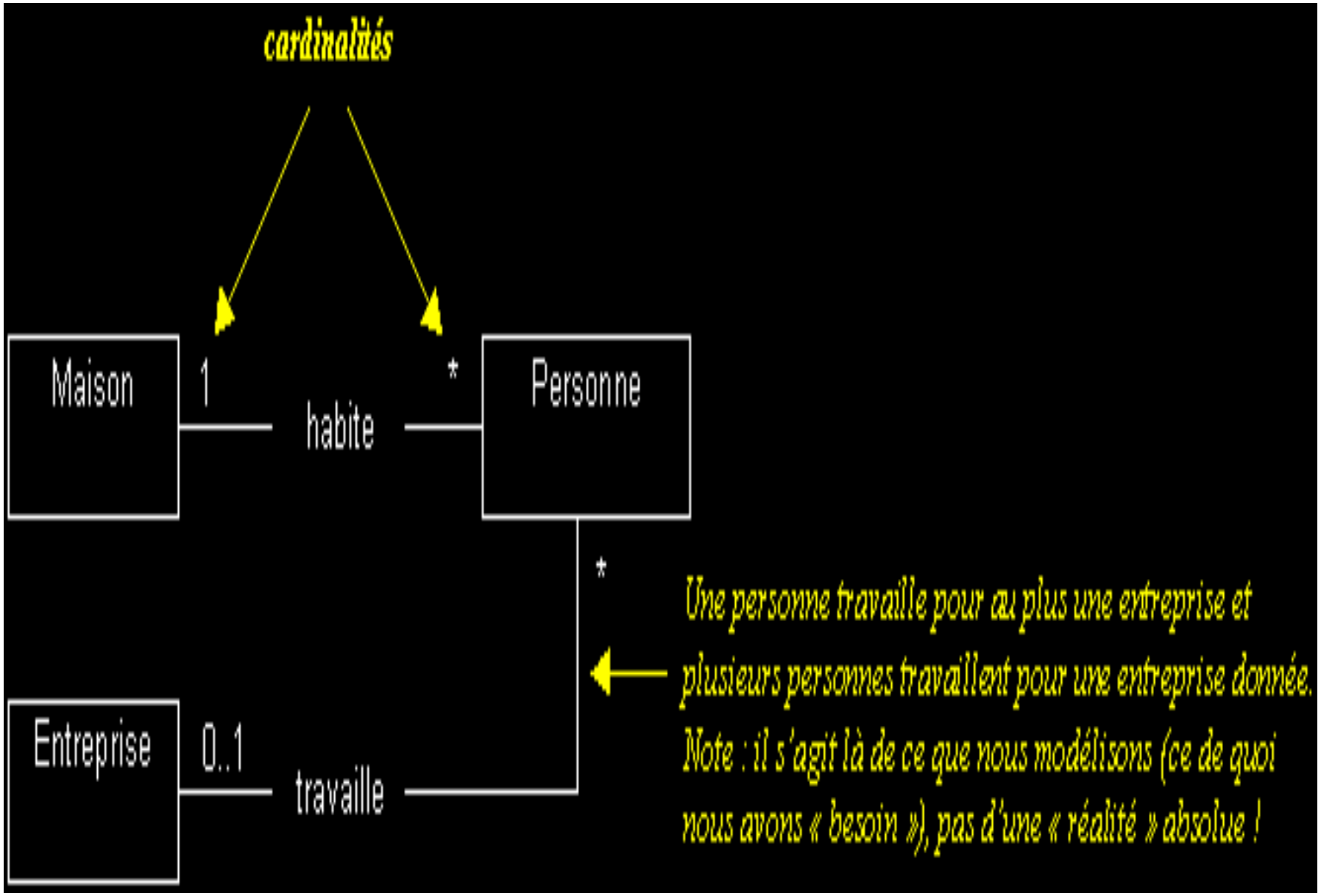
II-5 Multiplicité ou cardinalité

précise le nombre d'instances qui participent à une relation.

Expression des cardinalités d'une relation en UML :

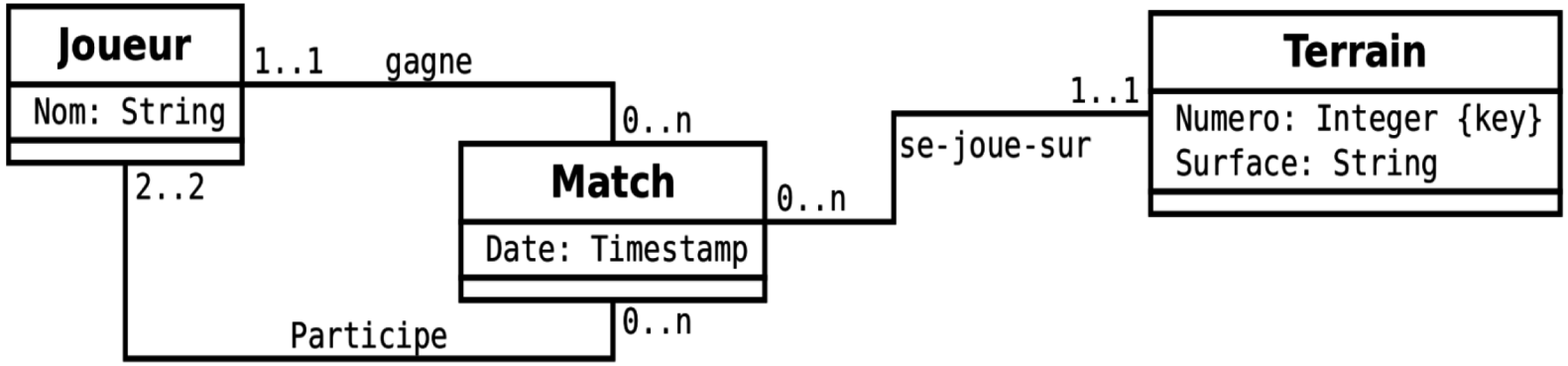
- **n** : exactement "n" (n, entier naturel > 0)
- **n..m** : de "n" à "m" (entiers naturels m ≥ n)
- ***** : plusieurs (équivalent à "0..n" et "0..*")
- **n..*** : "n" ou plus (n, entier naturel ou variable)

Exemple de cardinalité



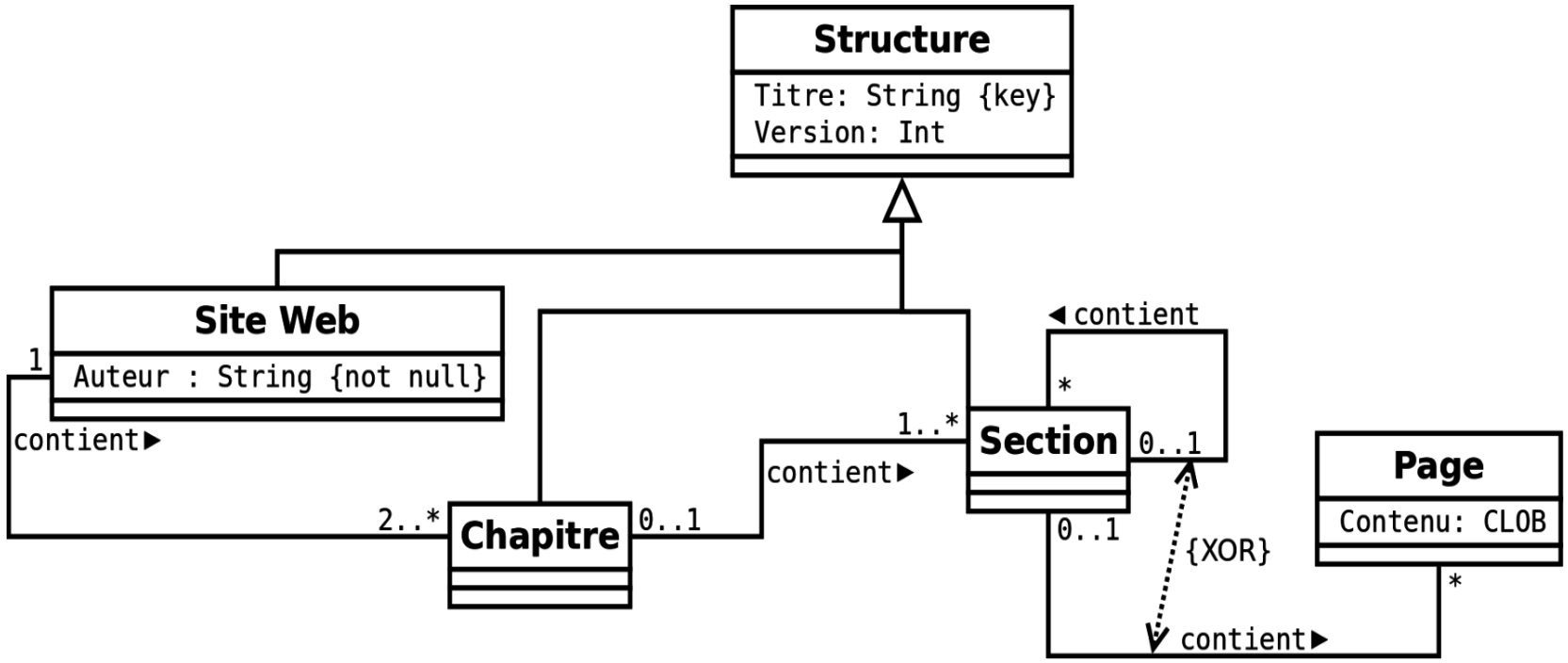
Exercice 1: Tournoi de tennis

Le schéma suivant représente les rencontres lors d'un tournoi de tennis. Quelles sont les assertions vraies selon ce schéma ?



- On peut jouer des matchs de double (ie 2 joueurs par équipe).
- Un joueur peut jouer 2 matchs en même temps.
- Un joueur peut gagner un match sans y avoir participé.
- Il peut y avoir deux matchs sur le même terrain à la même heure.
- Connaissant un joueur, on peut savoir sur quels terrains il a joué.

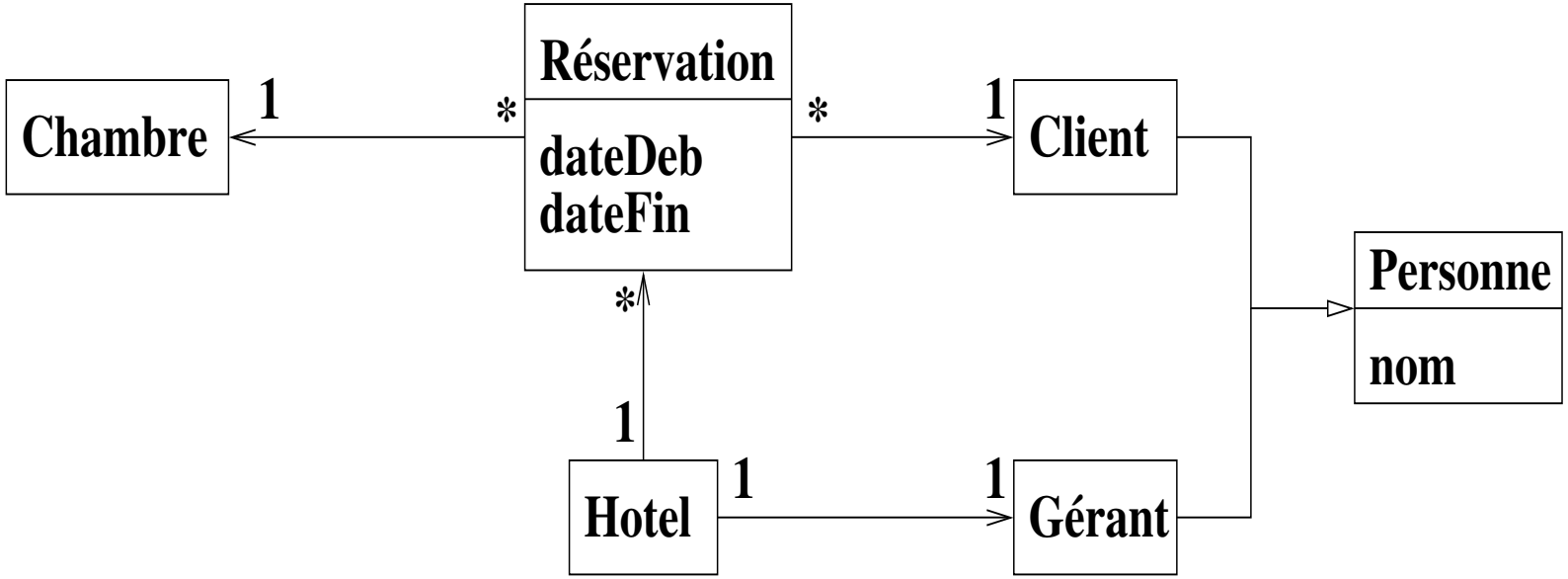
Exercice 2: site web



Vérifier si les assertions suivantes sont vraies selon le schéma ci-dessus:

- Chaque chapitre a un titre;
- Il est possible d'avoir un auteur diffèrent pour chaque chapitre;
- Toutes les sections contiennent au moins une section (ie une sous-section)
- Toutes les sections contiennent au moins une page;
- Tous les sites Web contiennent au moins une page.

Exercice 3: Gestion Hôtel



- Ce modèle permet-il de connaître les chambres d'un hôtel ?
- La connaissance d'une chambre occupée à un moment donnée permet-elle d'identifier son occupant ?
- Le modèle permet-il de changer de gérant d'un hôtel sans perdre des données ?

Pour chacune des questions faire une proposition qui prend en compte le besoin exprimé.
Donner le modèle physique correspondant au modèle initial et à chacun des modèles modifiés.

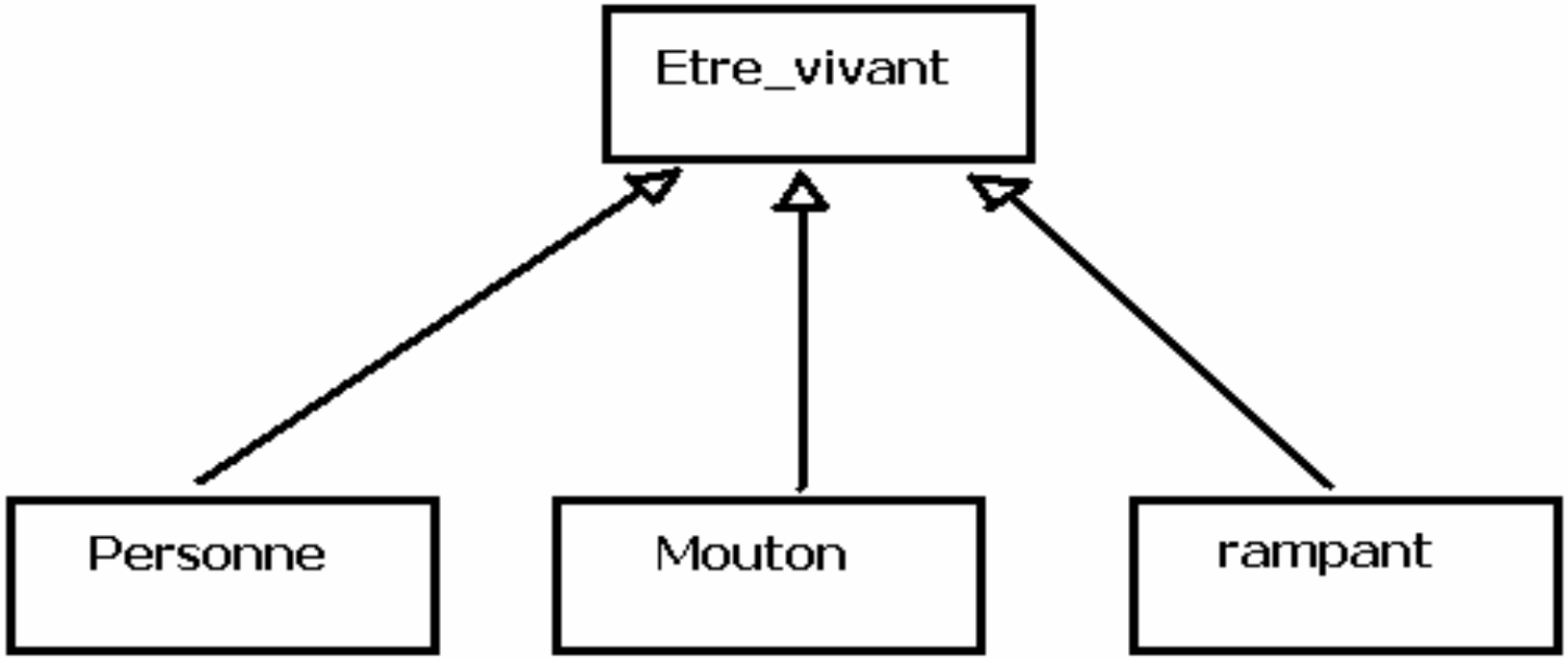
II-6 Héritage

- L'héritage est un mécanisme de transmission des propriétés d'une classe (ses attributs et méthodes) vers une sous-classe.

- L'héritage peut être simple ou multiple.

- L'héritage évite la duplication et encourage la réutilisation.

II-6-1 Exemple1 d'héritage



Implémentation:
Classe abstraite Forme et classe dérivée Rectangle

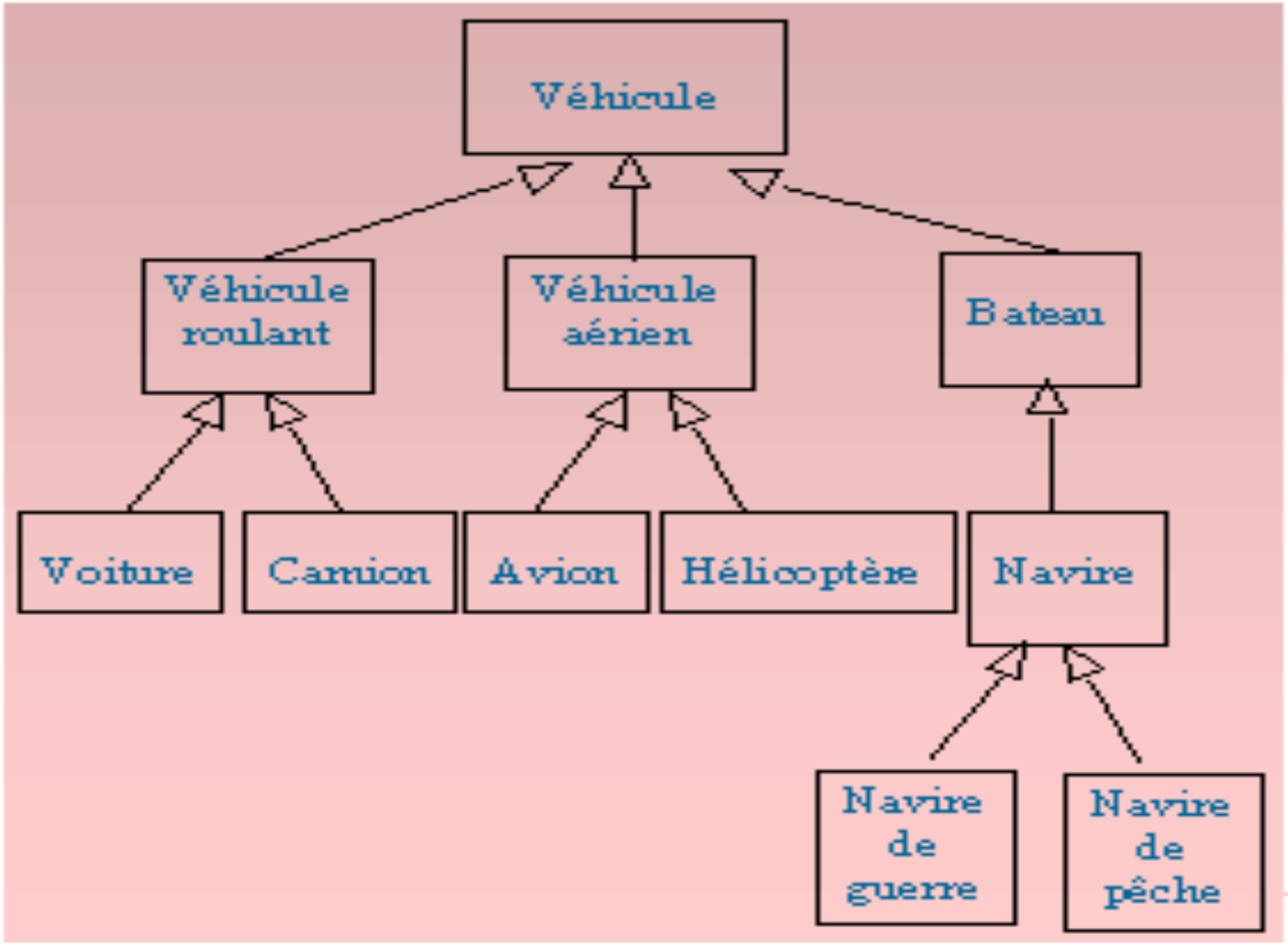
EN C++:

```
abstract class Forme
    {
            int x, y;   // coordonnée repère de la forme
            abstract float getSurface();
            abstract void draw();
    }

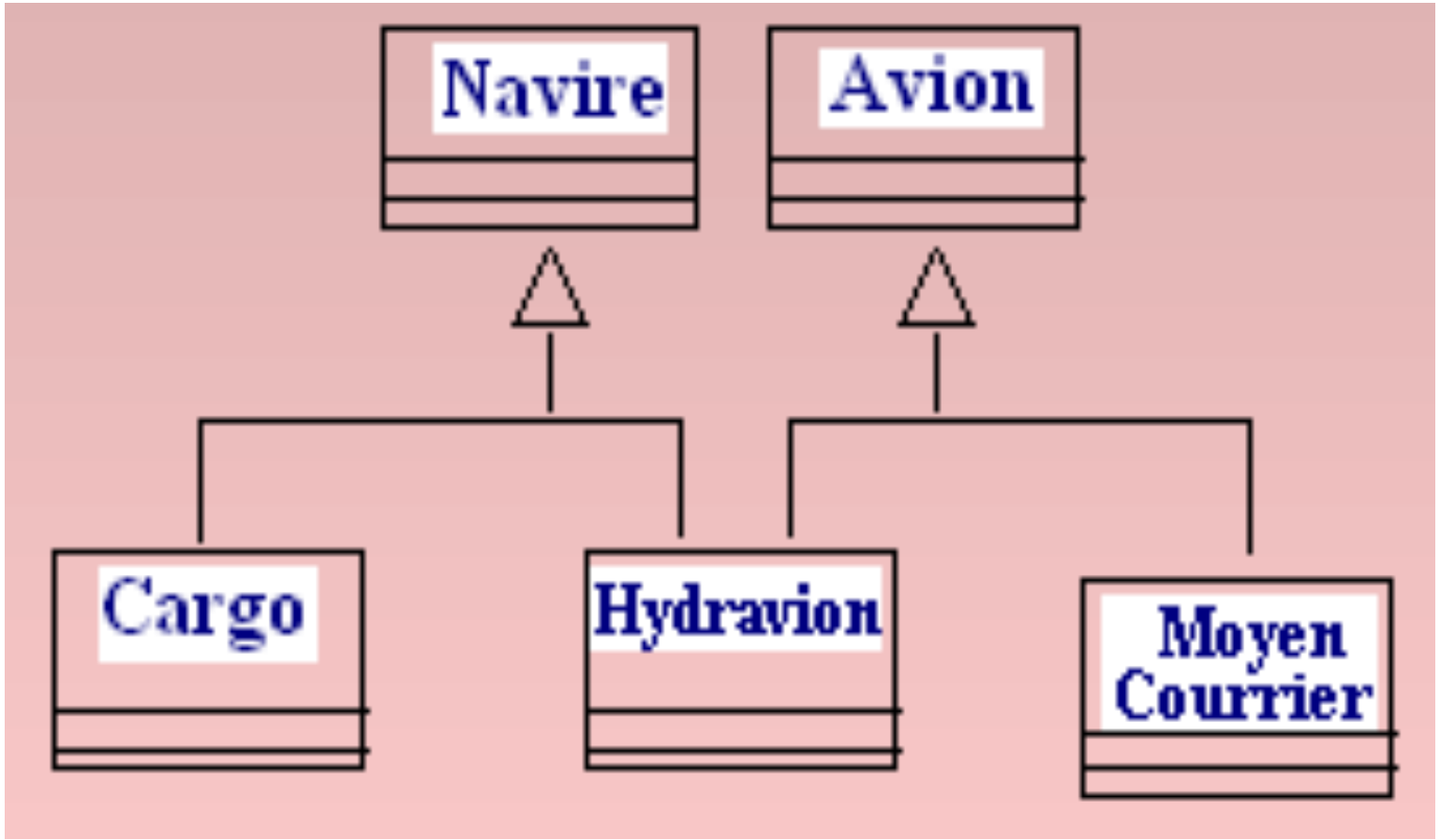
public class Rectangle extends Forme
{
    int longueur, largeur;
    public float getSurface()
    {
            return (float) longueur * largeur; // définition du corps
    }
    public void draw()
    { … // dessiner le rectangle // définition du corps

    }
}
```

II-6-1 Exemple 2 d'héritage (suite)



II-6-1 Exemple 3 d'héritage multiple (suite)



II-6-2 Héritage et redéfinition

La redéfinition d'une méthode héritée peut se faire de deux manières différentes:

par **réutilisation**

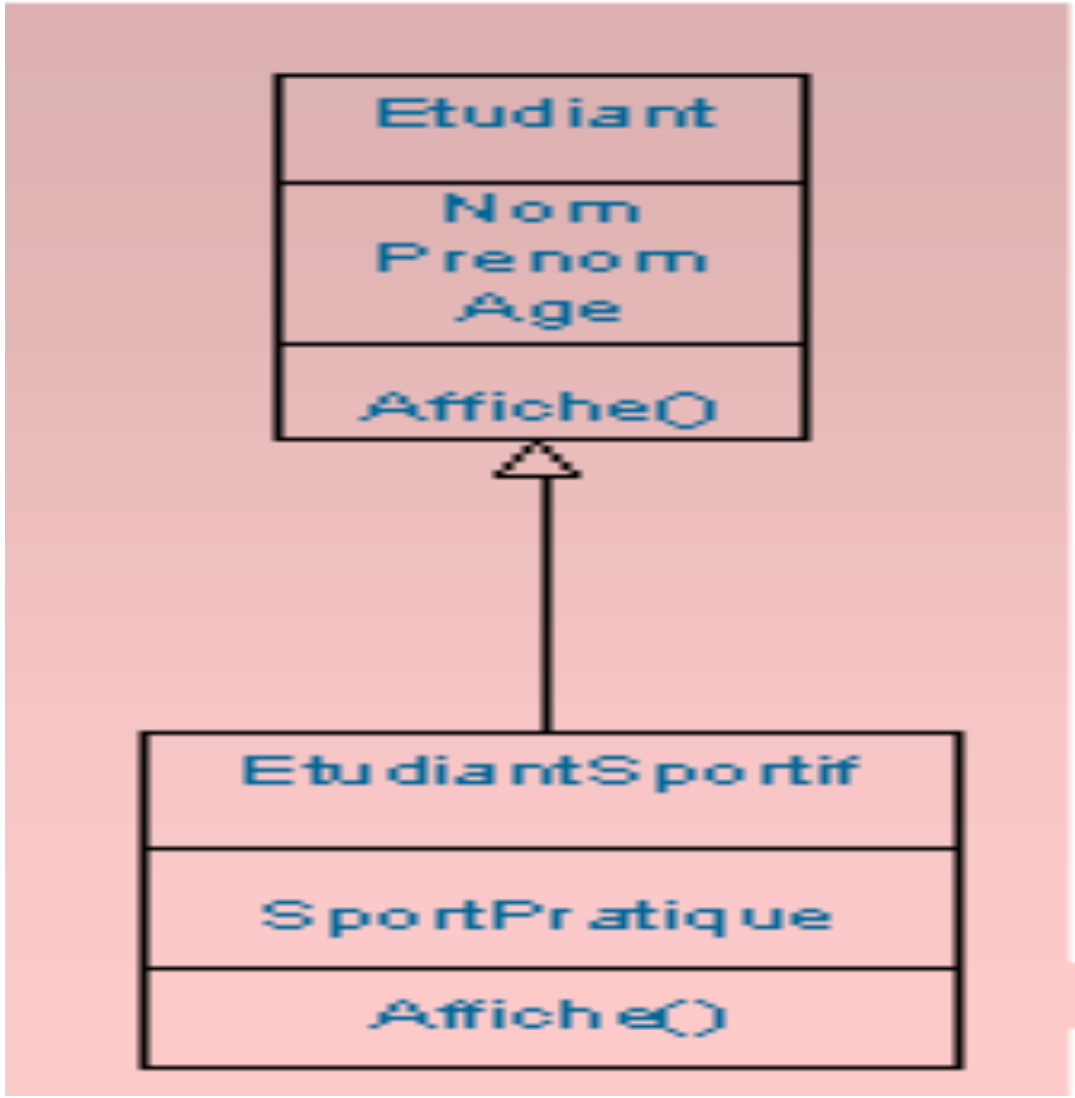
par **substitution**

II-6-2 Héritage et redéfinition (suite)

- par **réutilisation**

Spécialiser la méthode héritée en mettant une implémentation qui utilise celle héritée

Exemple de redéfinition par **réutilisation**

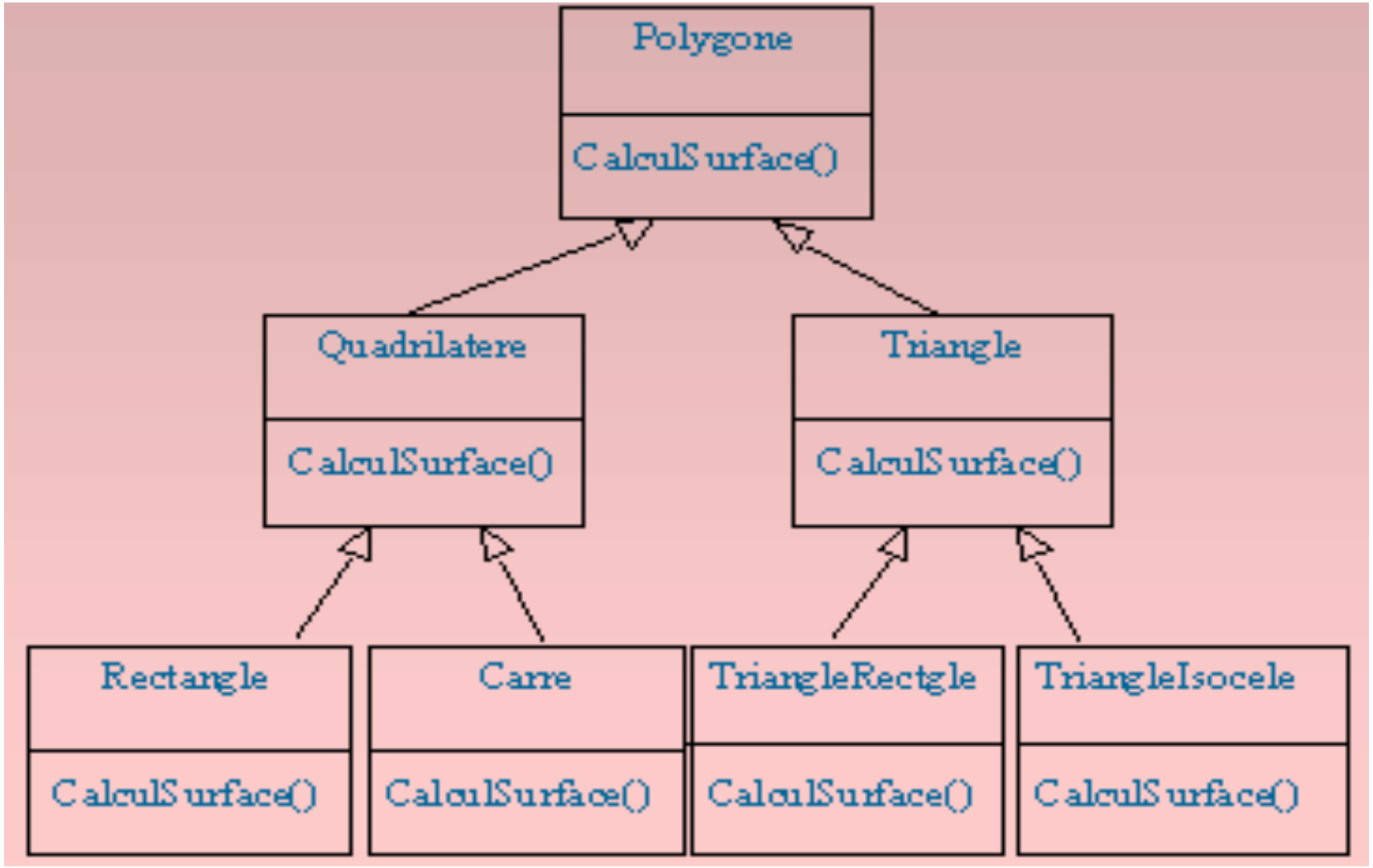


II-6-2 Héritage et redéfinition (suite)

par **substitution**

On remplace complètement la méthode héritée par une nouvelle implémentation

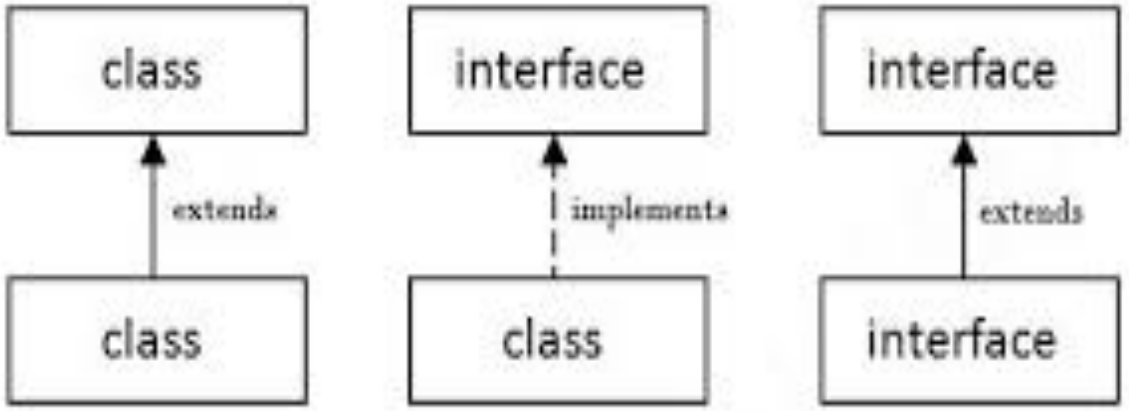
Exemple de redéfinition par **substitution**



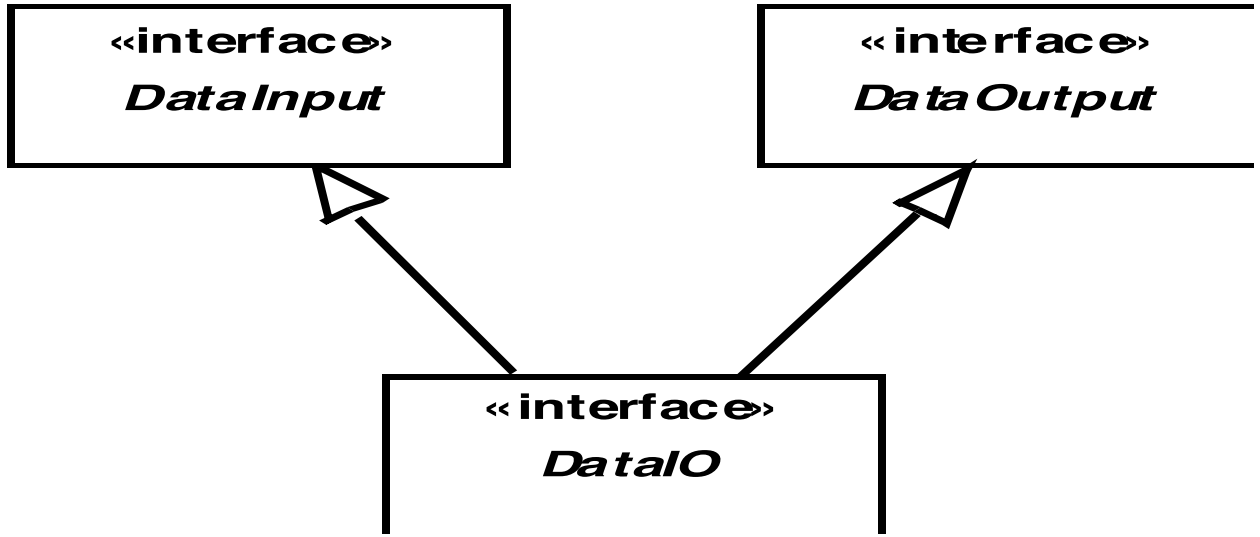
II-6-3 Relation d'Héritage et Relation de Réalisation

- **Héritage (extends):**
  - Entre **Classes** ou entre **Interfaces**

- **Relation de réalisation (implements):**
  - Entre **classe** et **Interface: une classe peut réaliser (ou implémenter) une ou plusieurs Interfaces**



Héritage multiples entre Interfaces




```
interface DataIO extends DataInput, DataOutput {...}
```

Ainsi DataIO contient les methods de DataInput, DataOutput et celles ajoutées

II-7 Généralisation/Spécialisation entre classes

- La spécialisation et la généralisation permettent de construire des hiérarchies de classes

II-7-1 Généralisation

- **Démarche ascendante, qui consiste à capturer les particularités communes d'un ensemble d'objets, issus de classes différentes.**

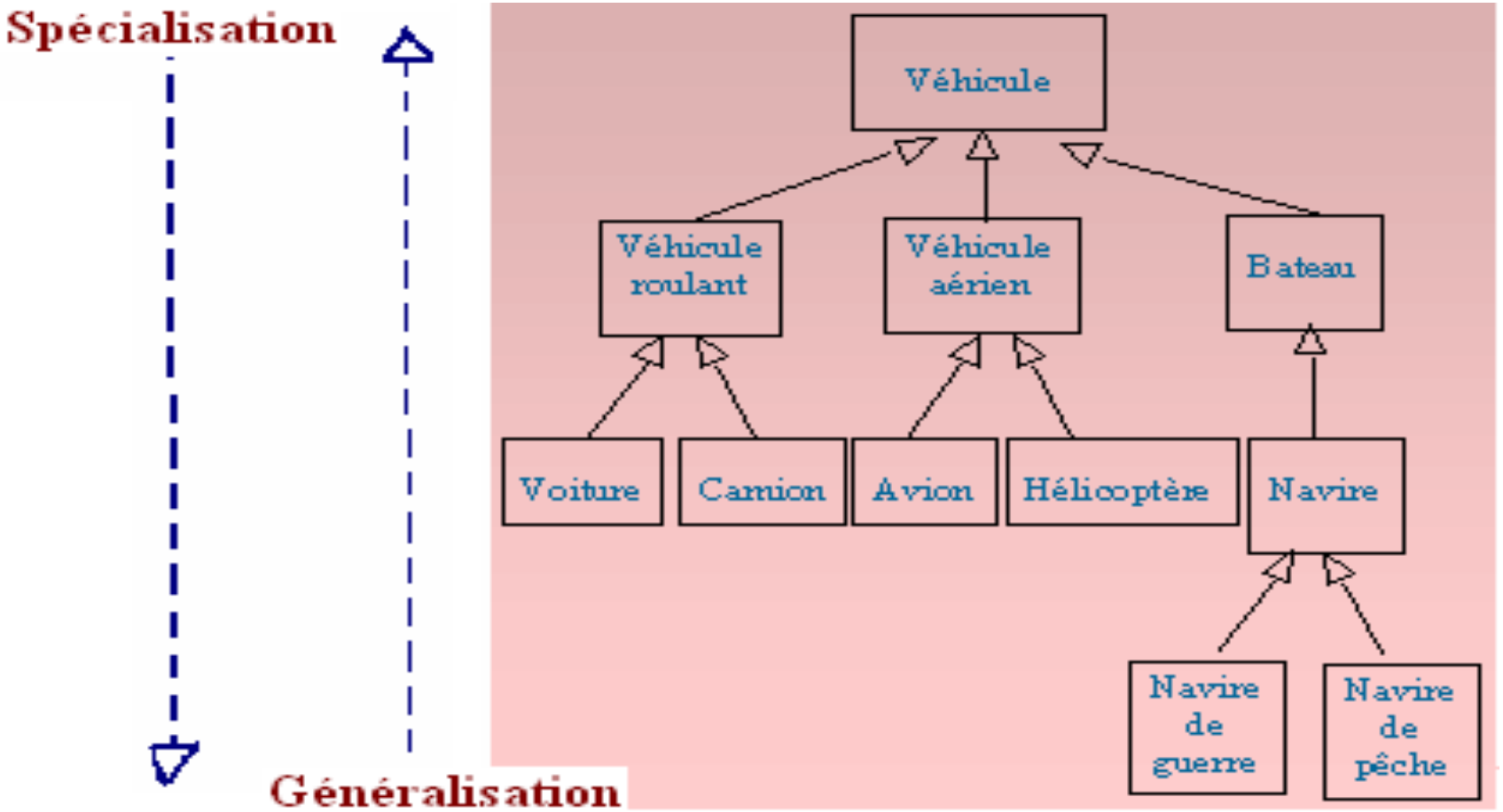
- **Consiste à factoriser les propriétés d'un ensemble de classes, sous forme d'une super-classe, plus abstraite (permet de gagner en généricité).**

II-7-2 Spécialisation

- **Démarche descendante, qui consiste à déterminer les particularités d'un ensemble d'objets, non discriminés par les classes déjà identifiées.**

- **Une classe peut être spécialisée en d'autres classes, ses sous-classes, afin d'y ajouter des caractéristiques spécifiques ou d'en adapter certaines (permet l'extension du modèle par réutilisation de classe déjà existante).**

II-7-3 Exemple de Généralisation/Spécialisation



II-8 Polymorphisme

Le polymorphisme représente la faculté d'une méthode à pouvoir s'appliquer à des objets de classes différentes, qui héritent d'une même classe de base (super-classe).

Le polymorphisme ajoute de l'intelligence dans nos code.

Exemple de Polymorphisme

